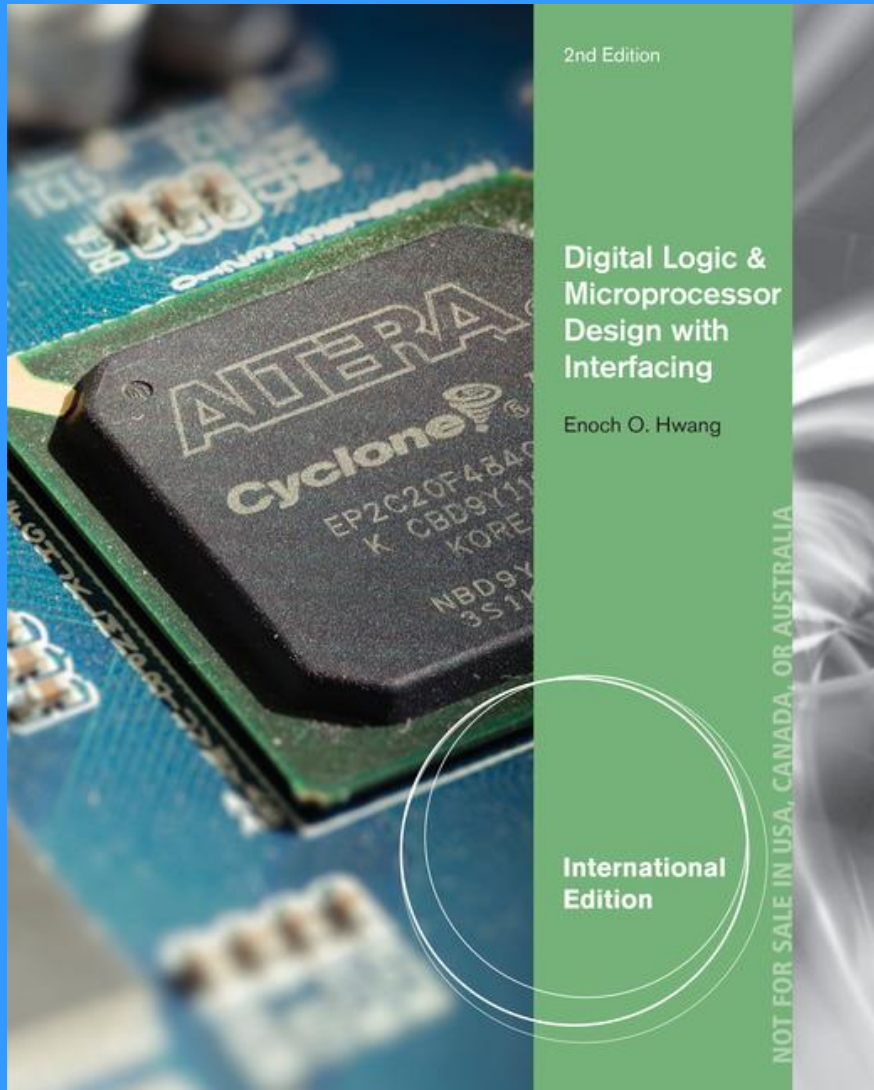




Chapter 5 Sequential Circuits

Sequential Circuits

- The outputs of sequential circuits are dependent on both the inputs and the past history
- In the car security system
 - The siren is turned on when the master switch is on and someone opens the door
 - If you close the door afterwards, the siren is turned off
 - To be more useful, if the siren has been turned on, then we want the siren to remain on even after the door closes afterwards
- This new car security system must depend on both the current inputs and the past history

Sequential Circuits

- Need to have memory elements to remember the history
- Uses the same logic gates as building blocks
- Difference is in how the gates are connected
- Have at least one feedback loop in the circuit
 - Output of a gate is connected back to the input

Sequential Circuits

- Latches and flip-flops are the basic memory elements
- A latch or a flip-flop can store one bit of information
- The value of this bit is referred to as the *state* of the latch or flip-flop
- Can be in one of two states: 0 or 1
- Change state when the value changes
- This value is always available at the output

In this Chapter

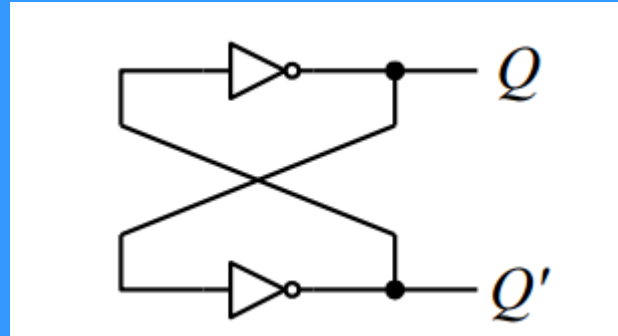
- Bistable Element
- SR Latch and SR Latch with enable
- D Latch and D Latch with enable
- Clock
- D Flip-Flop and D Flip-Flop with enable
- Description of a Flip-Flop
 - Characteristic Table
 - Characteristic Equation
 - State Diagram

In this Chapter

- Registers
- Register Files
- Memories
- Shift Registers
- Counters

Bistable Element

- Bistable element circuit



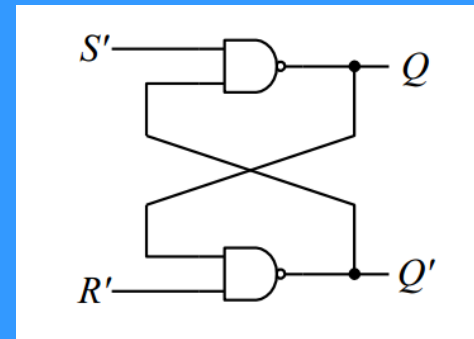
- Q and Q' are symmetrical: input, output signal (either)
- We arbitrarily pick Q as the state of the bistable element
- Two stable state: $Q = 0$ and $Q = 1$
- Memory: remembers its state
- Weakness: can't change its state

SR Latch

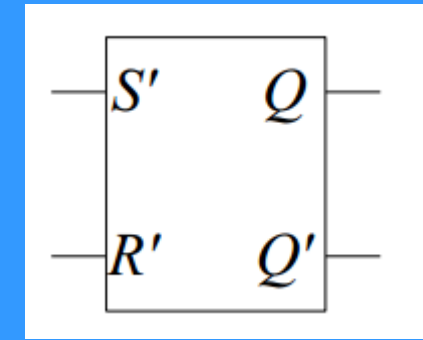
- Replace the two NOT gates in the bistable element with two NAND gates so that we can change the input

SR Latch

- SR Latch with two NAND gates
 - (a) circuit
 - (b) logic symbol
 - (c) truth table
 - (d) sample trace



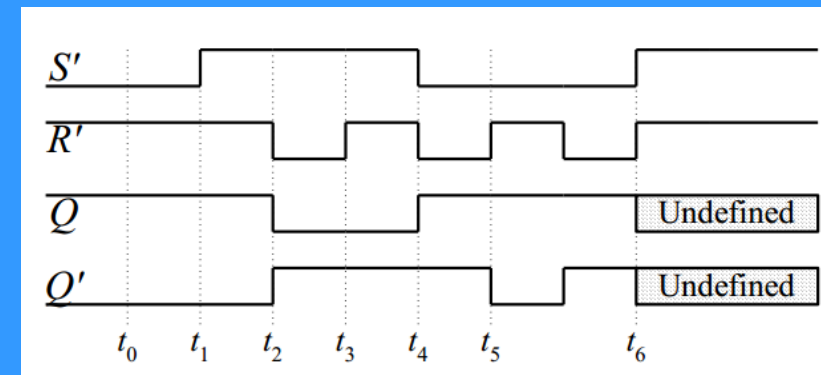
(a)



(b)

S'	R'	Q	Q_{next}	Q_{next}'
0	0	×	1	1
0	1	×	1	0
1	0	×	0	1
1	1	0	0	1
1	1	1	1	0

(c)

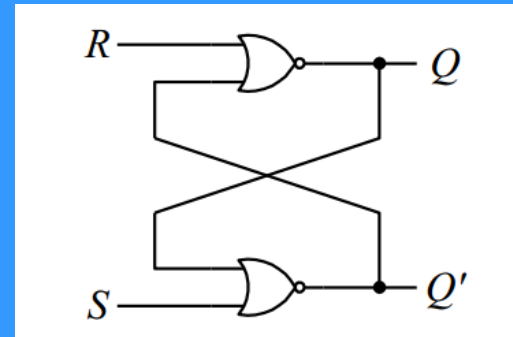


(d)

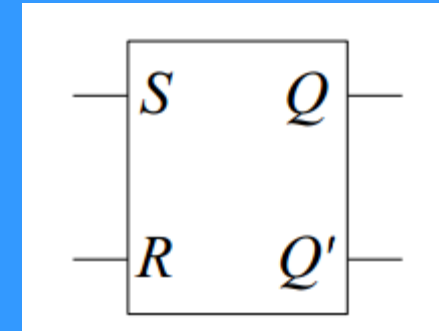
SR Latch

- SR Latch with two NOR gates

- (a) circuit
- (b) logic symbol
- (c) truth table



(a)



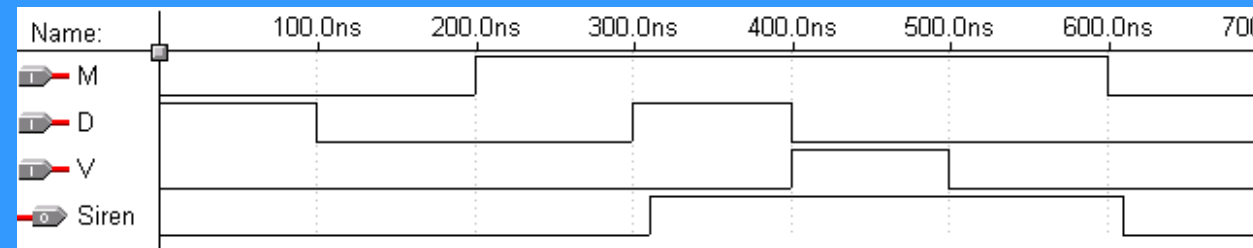
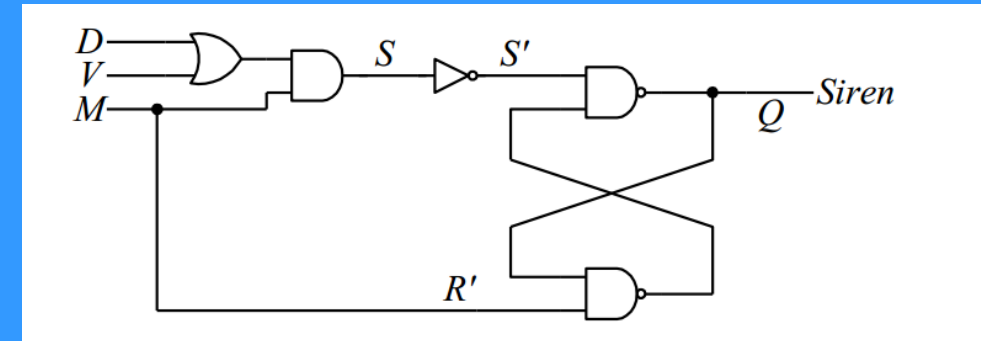
(b)

S	R	Q	Q_{next}	Q_{next}'
0	0	0	0	1
0	0	1	1	0
0	1	$\times$	0	1
1	0	$\times$	1	0
1	1	$\times$	0	0

(c)

Car Security System

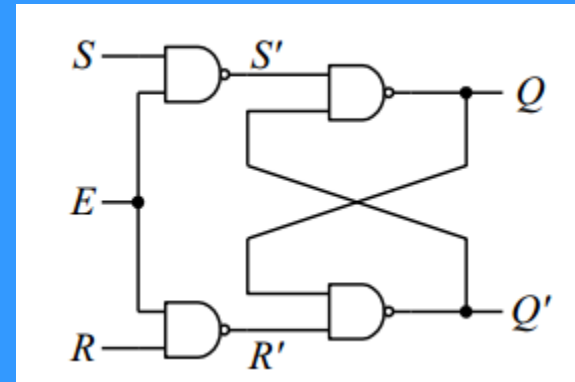
- Modified car security system circuit with memory (Sequential circuit)
- **Input: D = Door switch**
V = Vibration sensor
M = System on
Output: S = Siren
- Sample timing trace



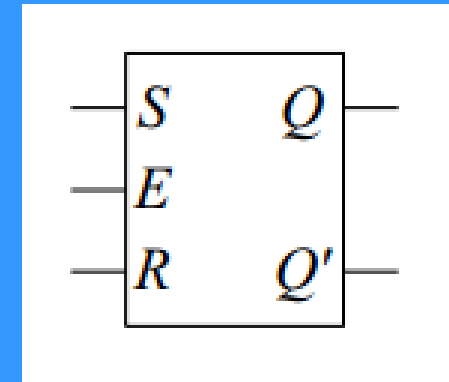
SR Latch with Enable

■ SR Latch with enable

- (a) circuit
- (b) logic symbol
- (c) truth table
- (d) sample trace



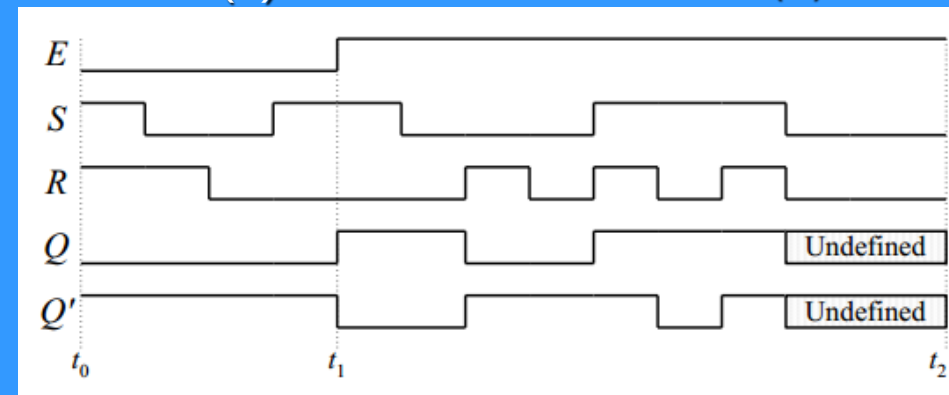
(a)



(b)

E	S	R	Q	Q_{next}	Q_{next}'
0	x	x	0	0	1
0	x	x	1	1	0
1	0	0	0	0	1
1	0	0	1	1	0
1	0	1	x	0	1
1	1	0	x	1	0
1	1	1	x	1	1

(c)



(d)

SR Latch

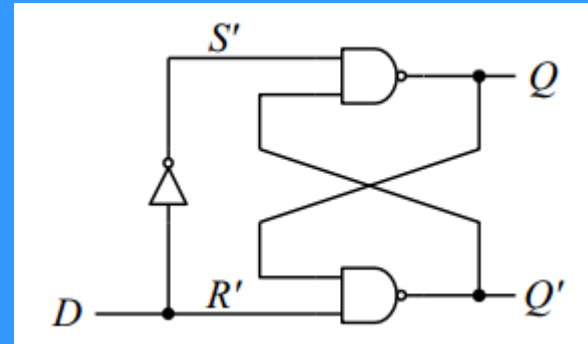
- Problem with the SR Latch
 - Assume that the symmetrical halves have exactly the same delay
 - Assume current value of Q and $Q' = 1$
 - What happens if we set S' and R' to a 1 at exactly the same time?

SR Latch

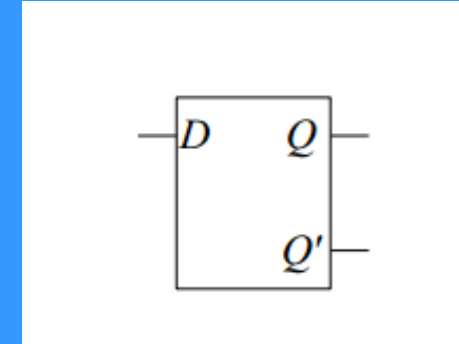
- Problem with the SR Latch
 - Will get an undefined state
 - Both S' and R' are de-asserted at *exactly* the same time
 - If both are never asserted at the same time, then they cannot be de-asserted at the same time
 - Simplest solution is to add an inverter between them

D Latch

- D Latch
- With NOT gate between R' and S'
 - (a) circuit
 - (b) logic symbol
 - (c) truth table
- What's lost?



(a)



(b)

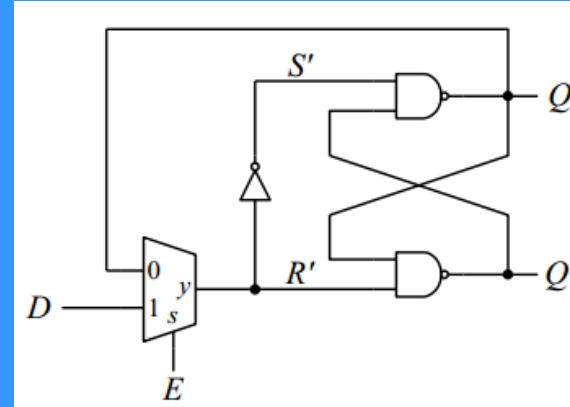
D	Q	Q_{next}	Q_{next}'
0	x	0	1
1	x	1	0

(c)

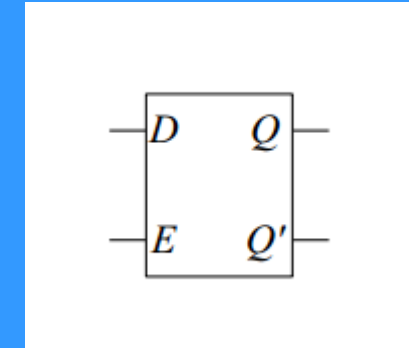
D Latch with Enable

■ D Latch with enable

- (a) circuit
- (b) logic symbol
- (c) truth table
- (d) sample trace



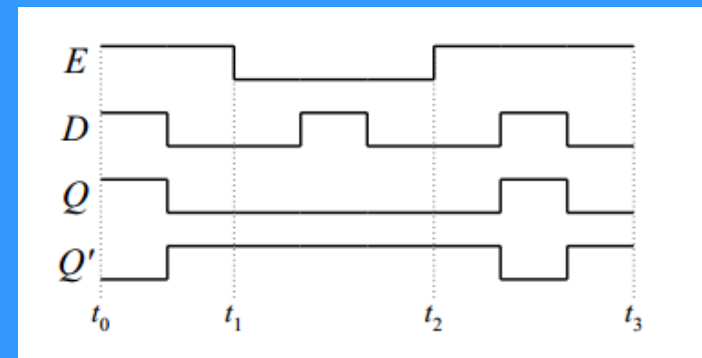
(a)



(b)

E	D	Q	Q_{next}	Q_{next}'
0	x	0	0	1
0	x	1	1	0
1	0	x	0	1
1	1	x	1	0

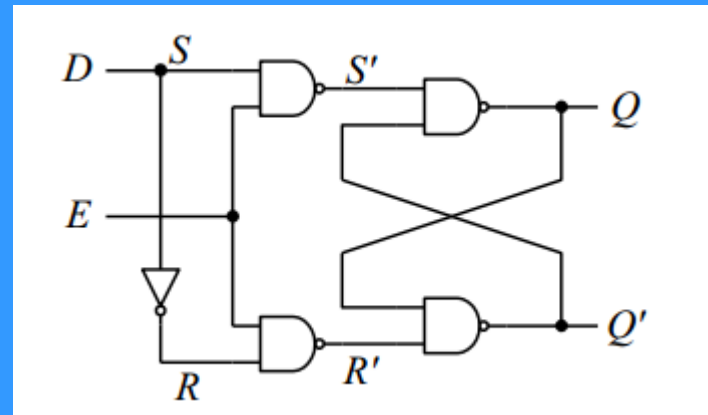
(c)



(d)

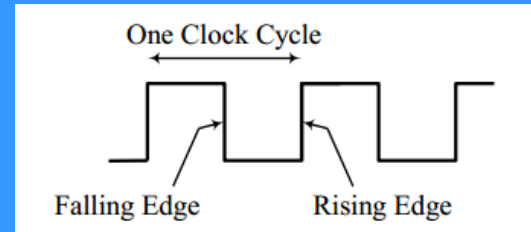
D Latch with Enable

- D latch with enable circuit using four NAND gates
- When $E = 0$, S' and R' are de-asserted, keep state
- When $E = 1$, becomes a D latch

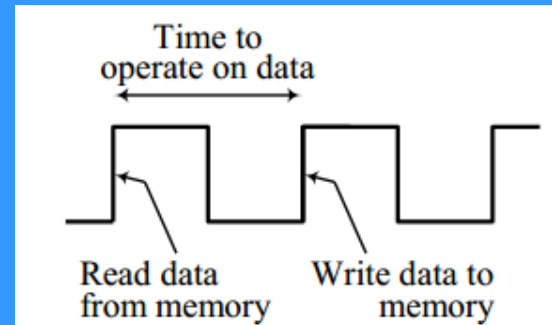


Clock

- Clock signal as the synchronizing signal
 - active edge: rising edge or falling edge
 - clock cycle
 - period

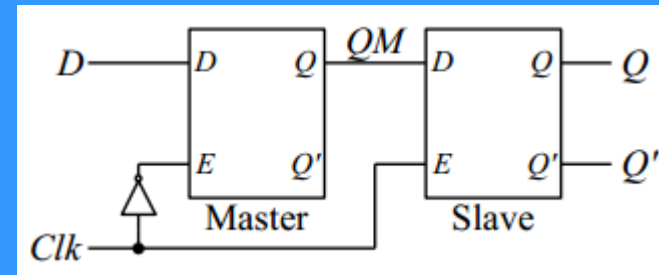


- Relationship between the clock period and the time to operate on the data

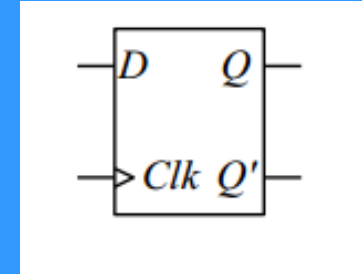


D Flip-Flop

- Master-slave positive edge-triggered D flip-flop
 - (a) circuit
 - (b) logic symbol
 - (c) operation table
 - (d) sample trace



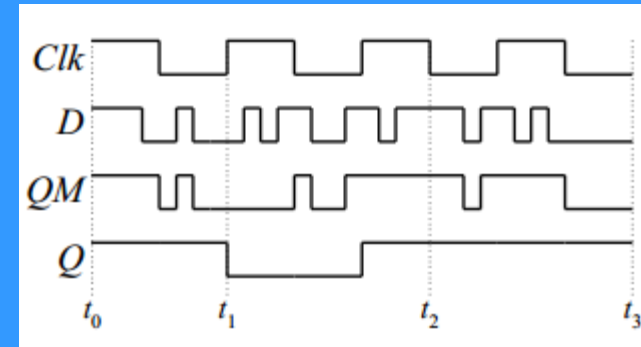
(a)



(b)

Clk	D	Q	Q_{next}	Q_{next}'
0	x	0	0	1
0	x	1	1	0
1	x	0	0	1
1	x	1	1	0
↑	0	x	0	1
↑	1	x	1	0

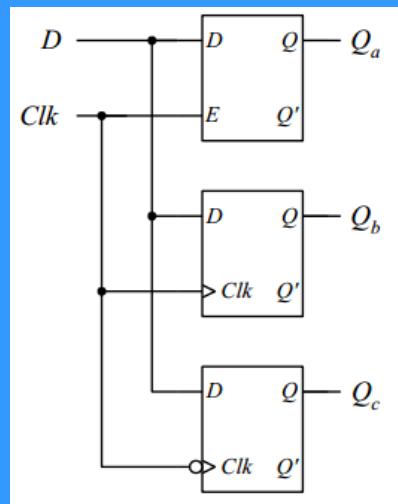
(c)



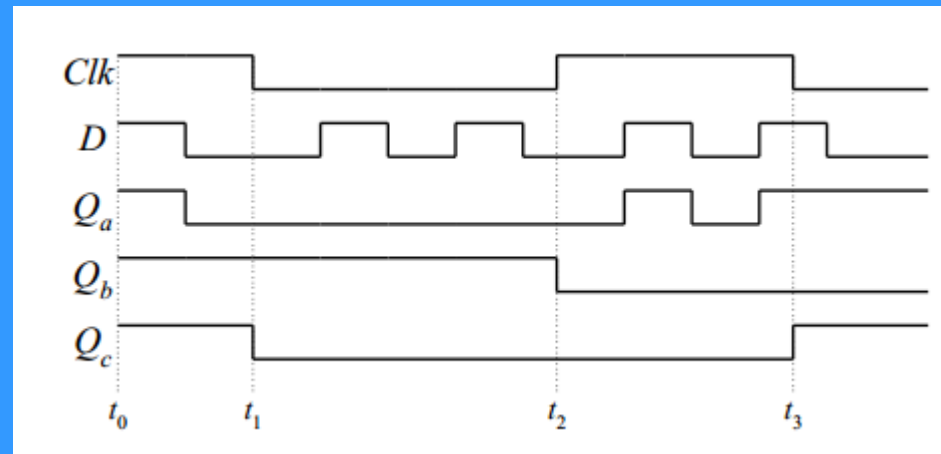
(d)

D Flip-Flop

- Comparison of
 - Gated latch
 - positive edge-triggered flip-flop
 - negative edge-triggered flip-flop



(a) circuit

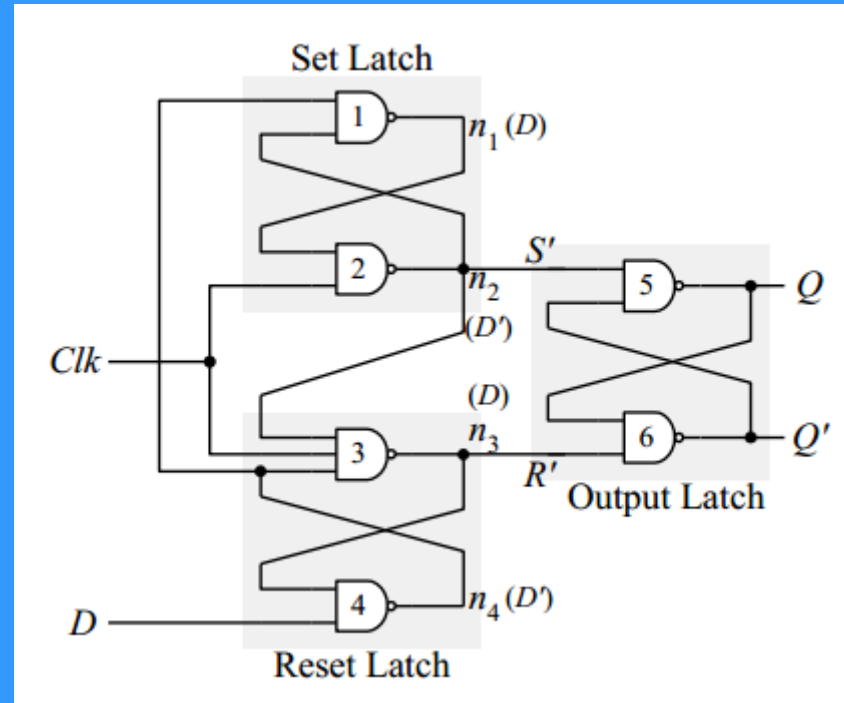


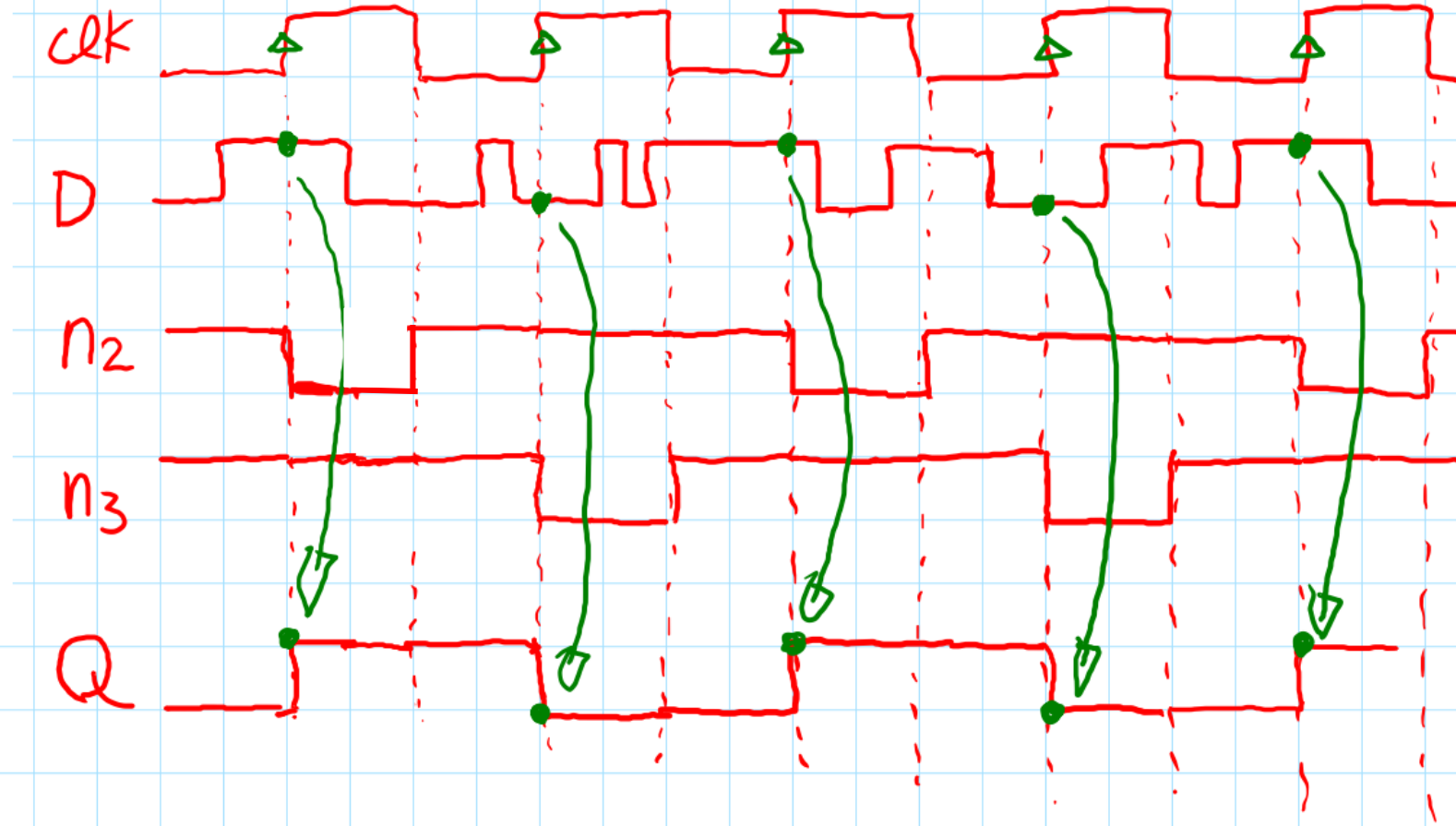
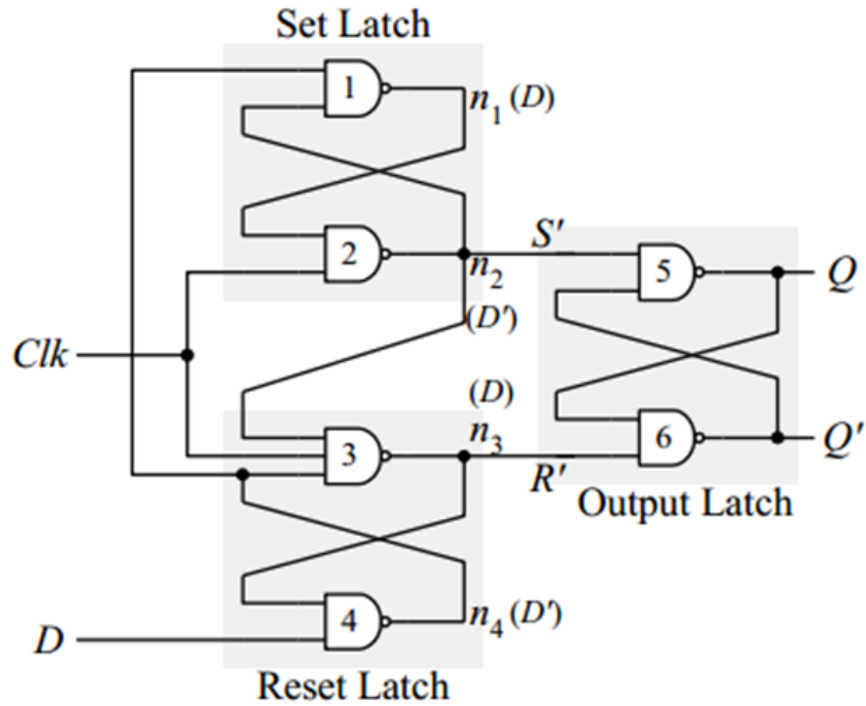
(b) sample trace

Quiz 5.1

*Alternative Smaller Circuit

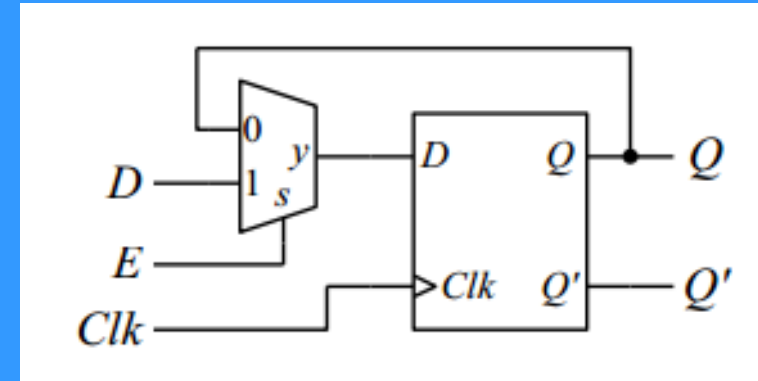
- Positive edge-triggered D flip-flop using SR latches





D Flip-Flop with Enable

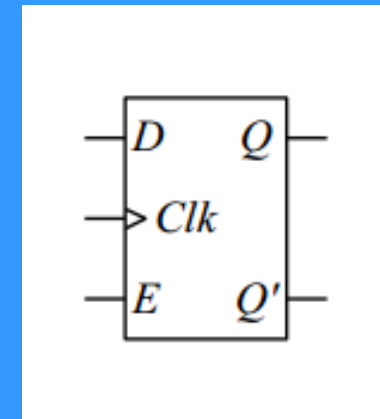
- D Flip-flop with enable
 - (a) circuit
 - (b) logic symbol
 - (c) operation table



(a)

Clk	E	D	Q	Q_{next}	Q_{next}'
0	x	x	0	0	1
0	x	x	1	1	0
1	x	x	0	0	1
1	x	x	1	1	0
↑	0	x	0	0	1
↑	0	x	1	1	0
↑	1	0	x	0	1
↑	1	1	x	1	0

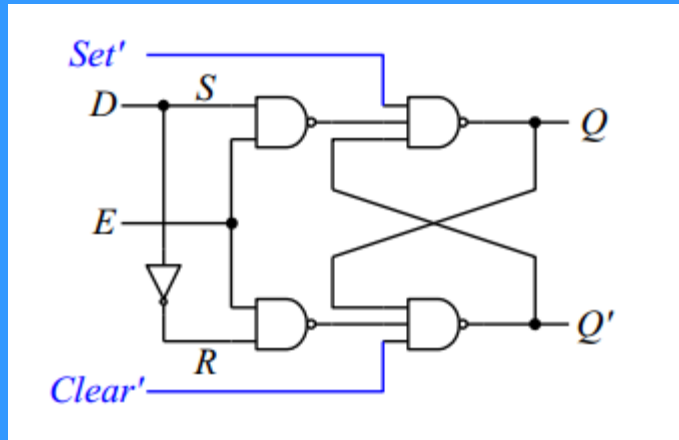
(b)



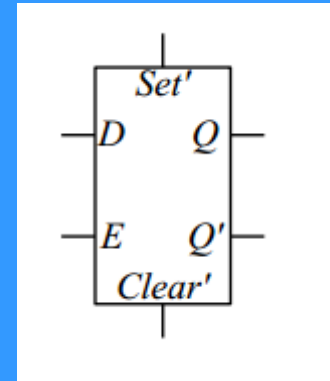
(c)

Asynchronous Inputs

- Storage elements with asynchronous inputs
 - (a) D latch with active-low set and clear
 - (b) logic symbol for (a)



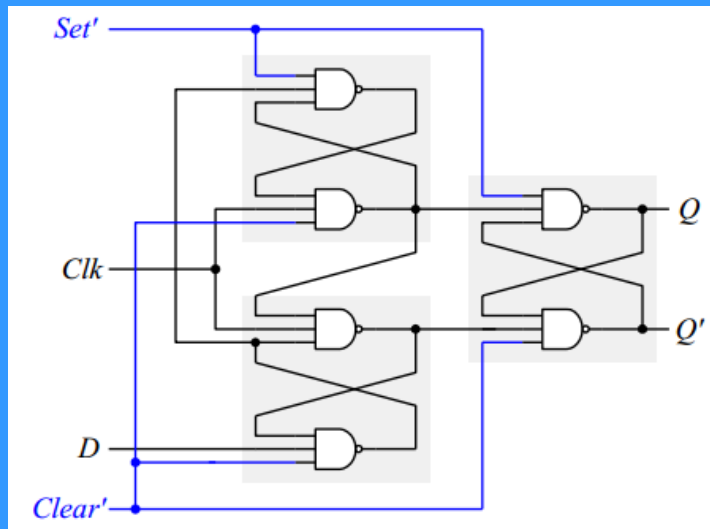
(a)



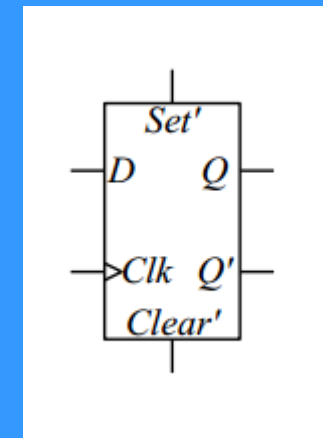
(b)

Asynchronous Inputs

- Storage elements with asynchronous inputs
 - (c) D edge-triggered flip-flop with active-low set and clear
 - (d) logic symbol for (c)



(c)



(d)

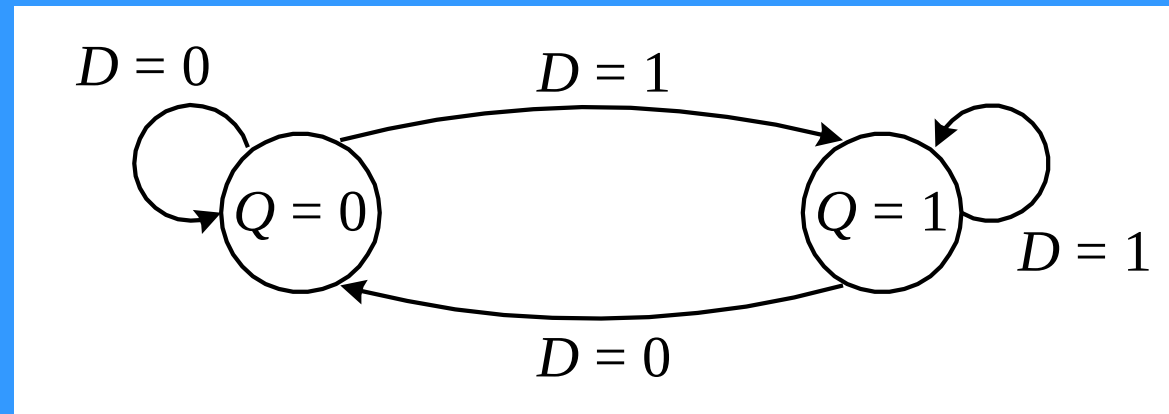
Description of a Flip-Flop

D	Q	Q_{next}
0	×	0
1	×	1

Characteristic Table

$$Q_{\text{next}} = D$$

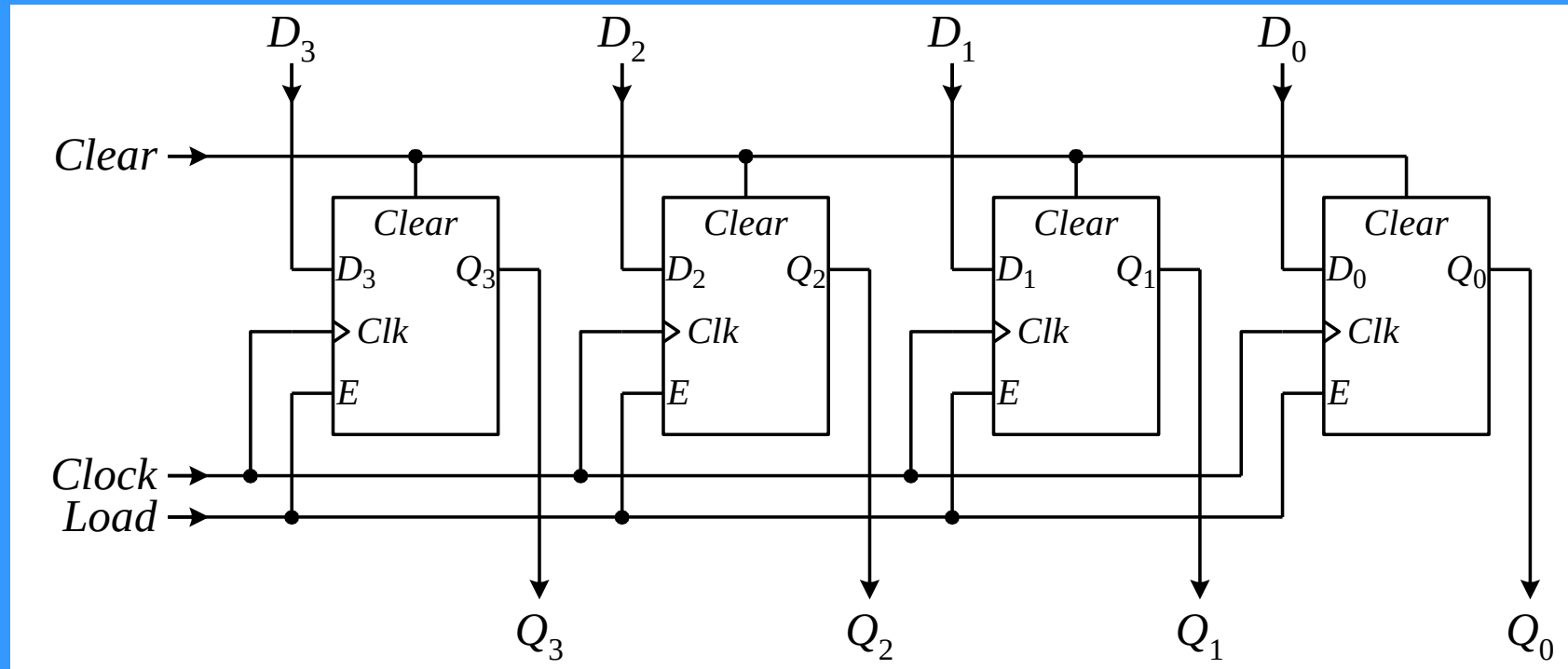
Characteristic Equation



State Diagram

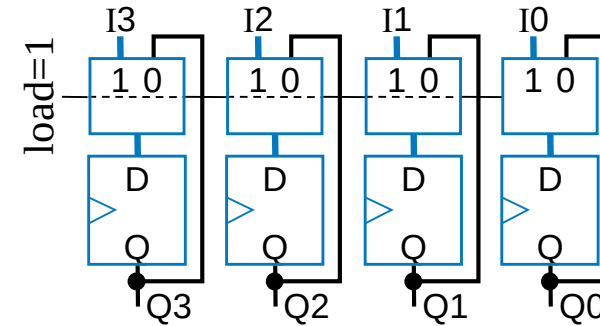
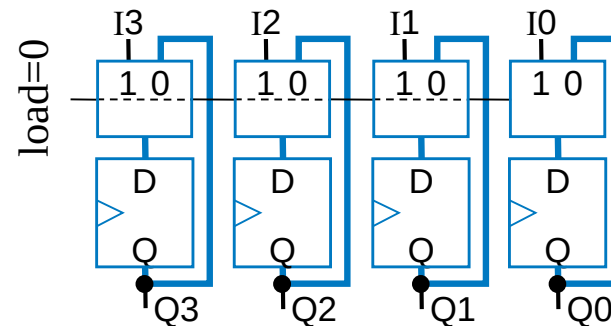
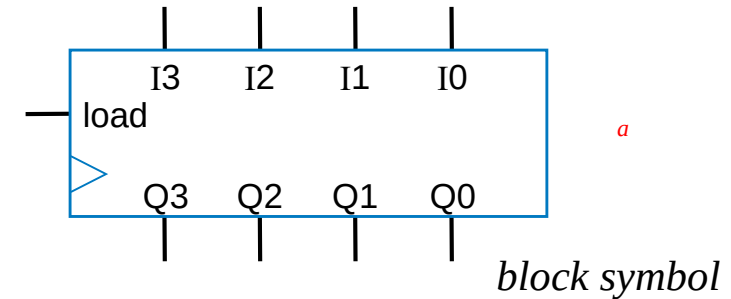
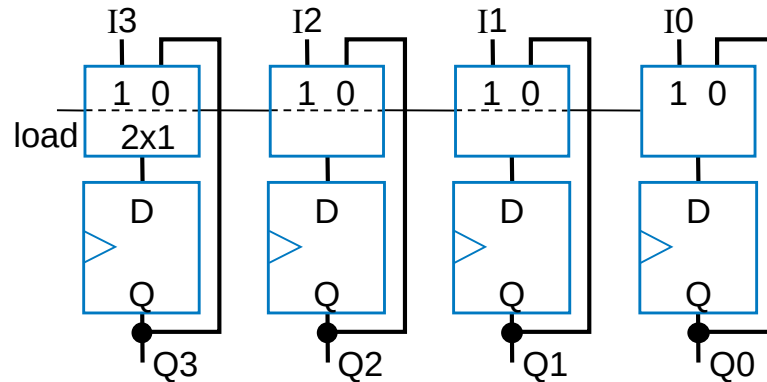
Register

- A 4-bit register with synchronous load and asynchronous clear



Register with Parallel Load

- Add 2x1 mux to front of each flip-flop
- Register's *load* input selects mux input to pass
 - load=0: existing flip-flop value; load=1: new input value

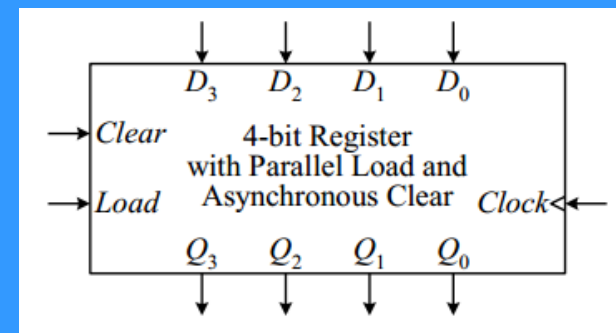


Register

- A 4-bit register with synchronous load and asynchronous clear

<i>Clear</i>	<i>Load</i>	Operation
1	×	Reset register to zero asynchronously
0	0	No change
0	1	Load in <i>D</i> value at rising clock edge

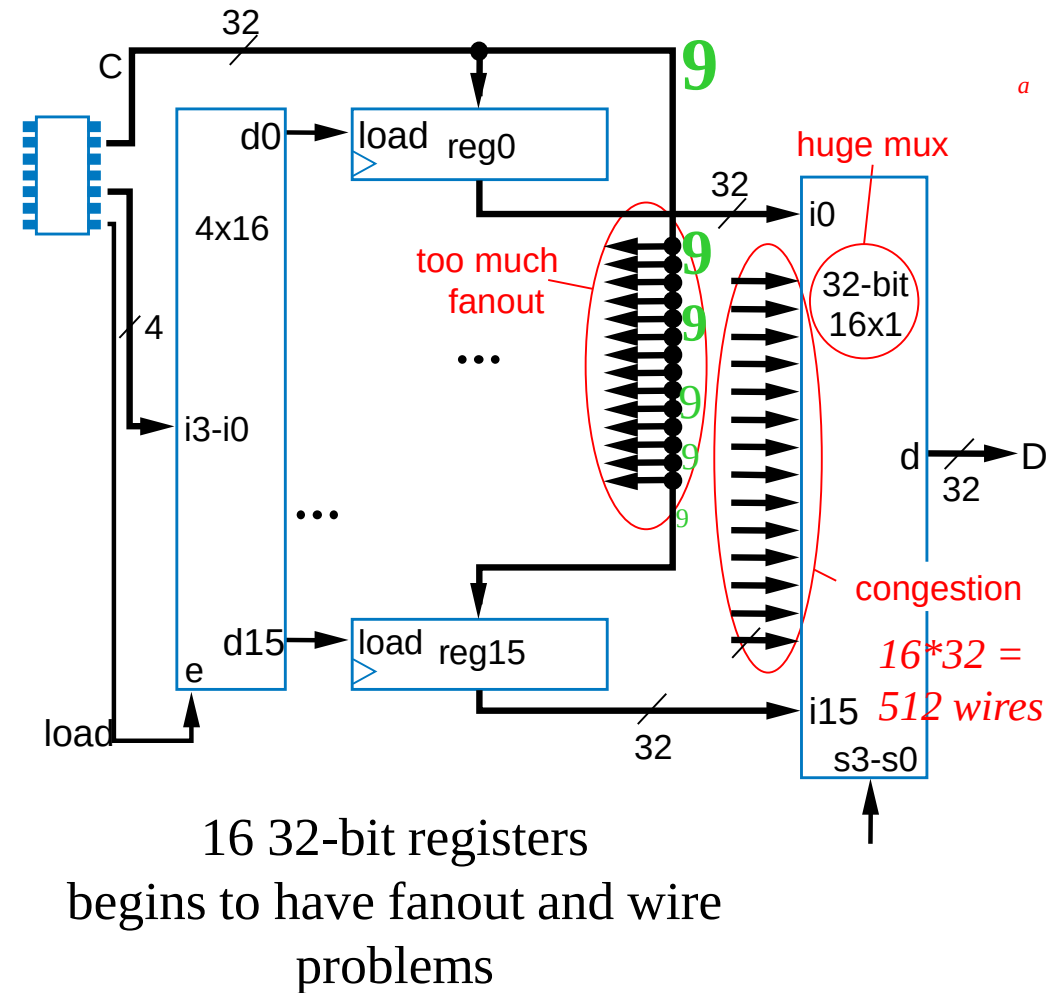
operation table



symbol

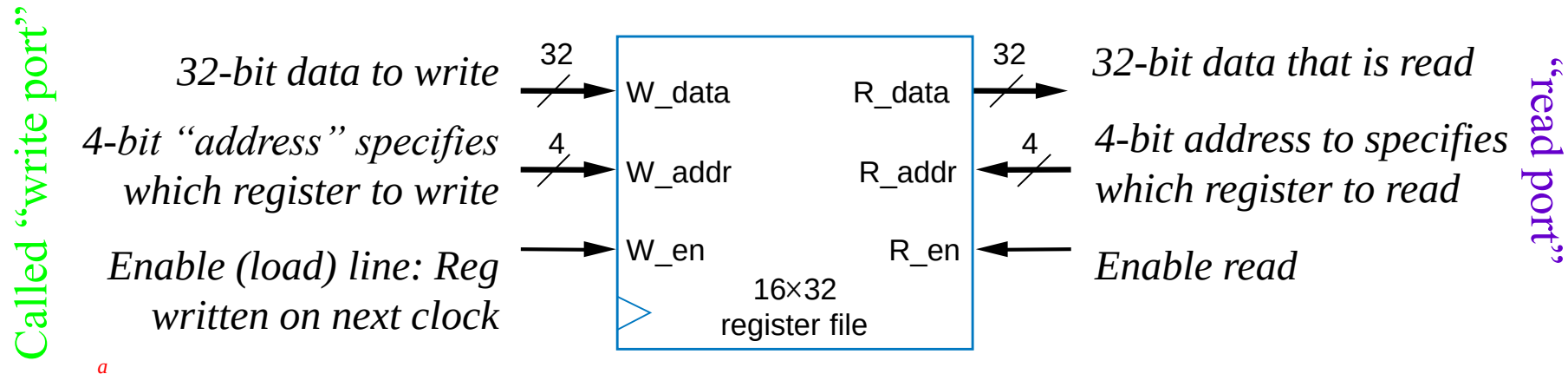
Register Files

- Accessing one of several registers is:
 - OK if just a few registers
 - Problematic when many
 - Ex: Earlier above-mirror display, with 16 registers
 - Much fanout (branching of wire): Weakens signal
 - Many wires: Congestion



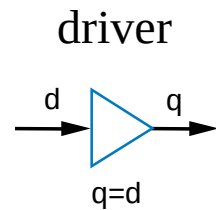
Register File

- **MxN register file:** Efficient design for one-at-a-time write/read of many registers
 - Consider 16 32-bit registers



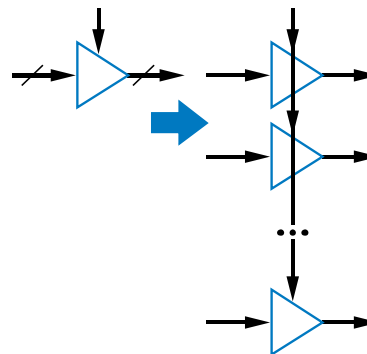
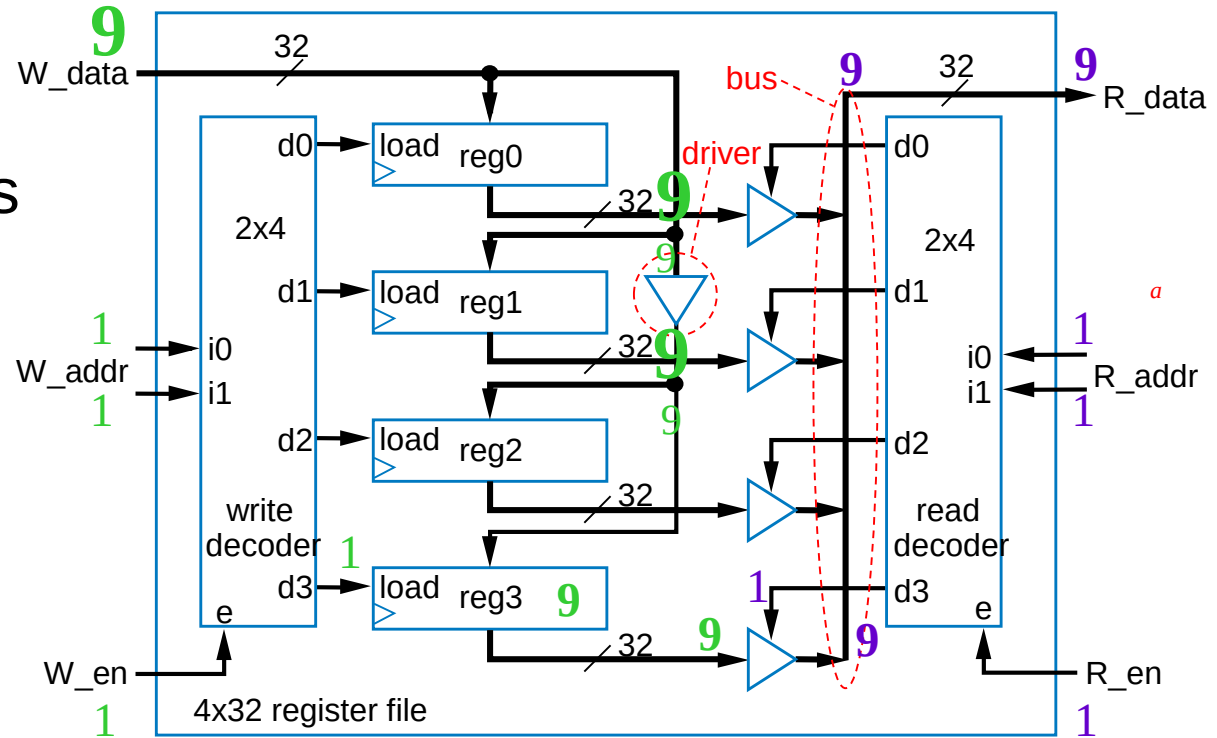
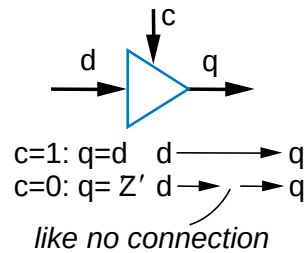
Register File

- Internal design uses drivers and bus



Boosts signal

three-state driver

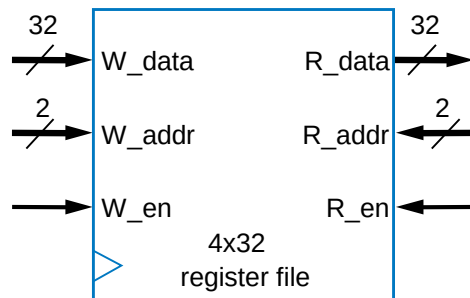
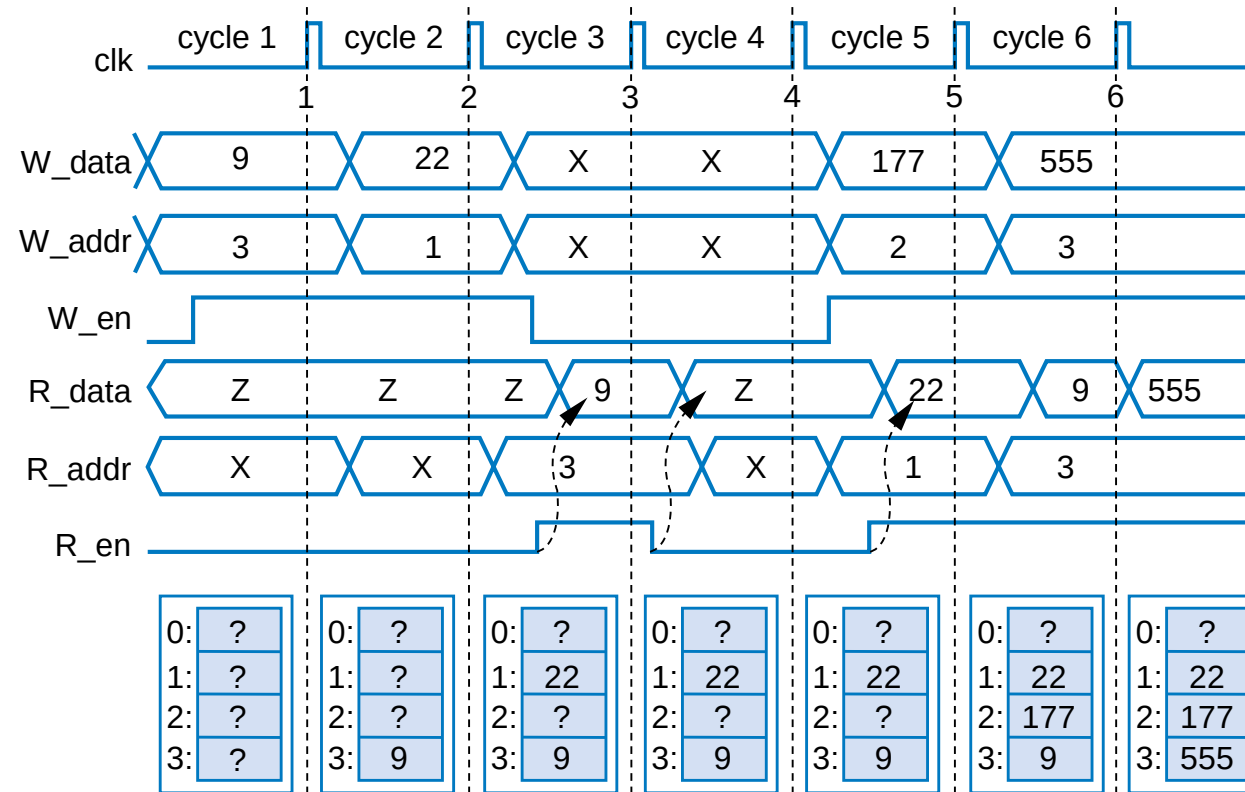


Note: Each driver in figure actually represents 32 1-bit drivers



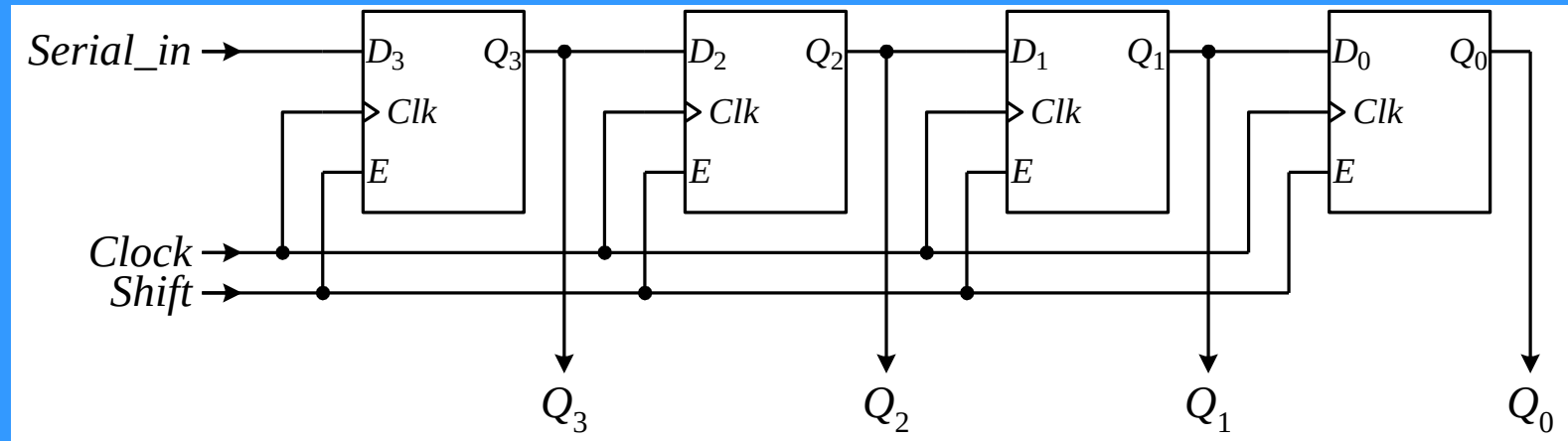
Register File Timing Diagram

- Can write one register and read one register each clock cycle
 - May be same register

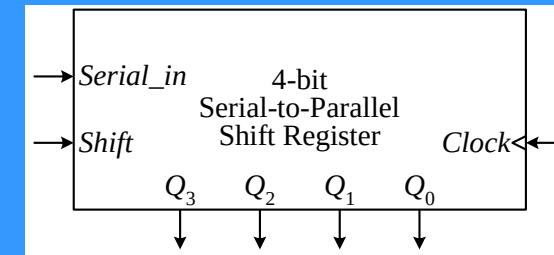


Shift Register

- For serial-to-parallel data conversion

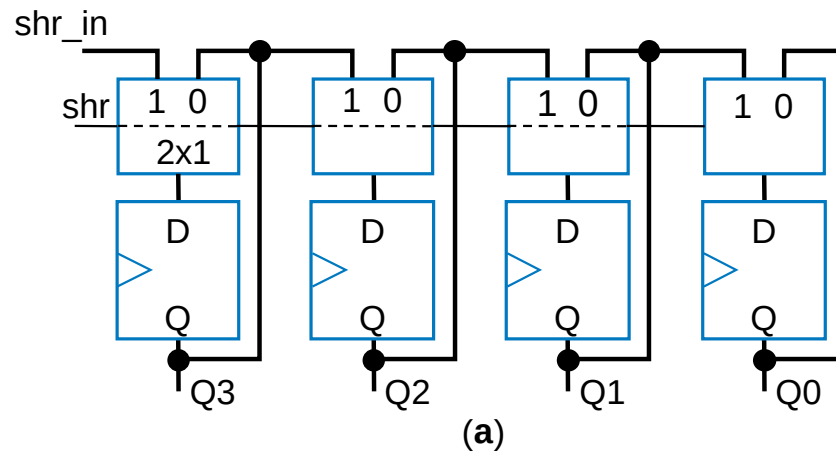


Shift	Operation
0	No change
1	One bit from <i>Serial_in</i> is shifted in

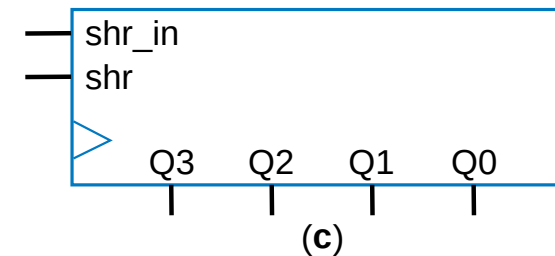
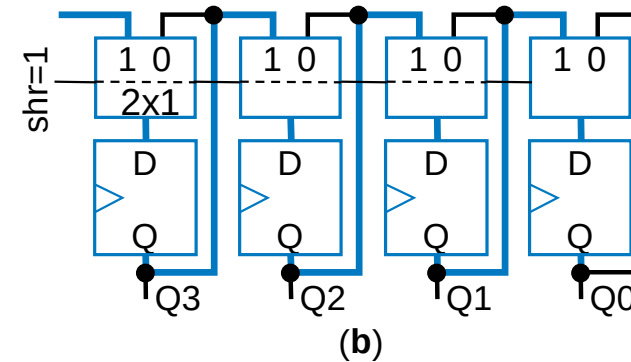


Shift Register

- To allow register to either shift or retain, use 2x1 muxes
 - shr: “0” means retain, “1” shift
 - shr_in: value to shift in
 - May be 0, or 1

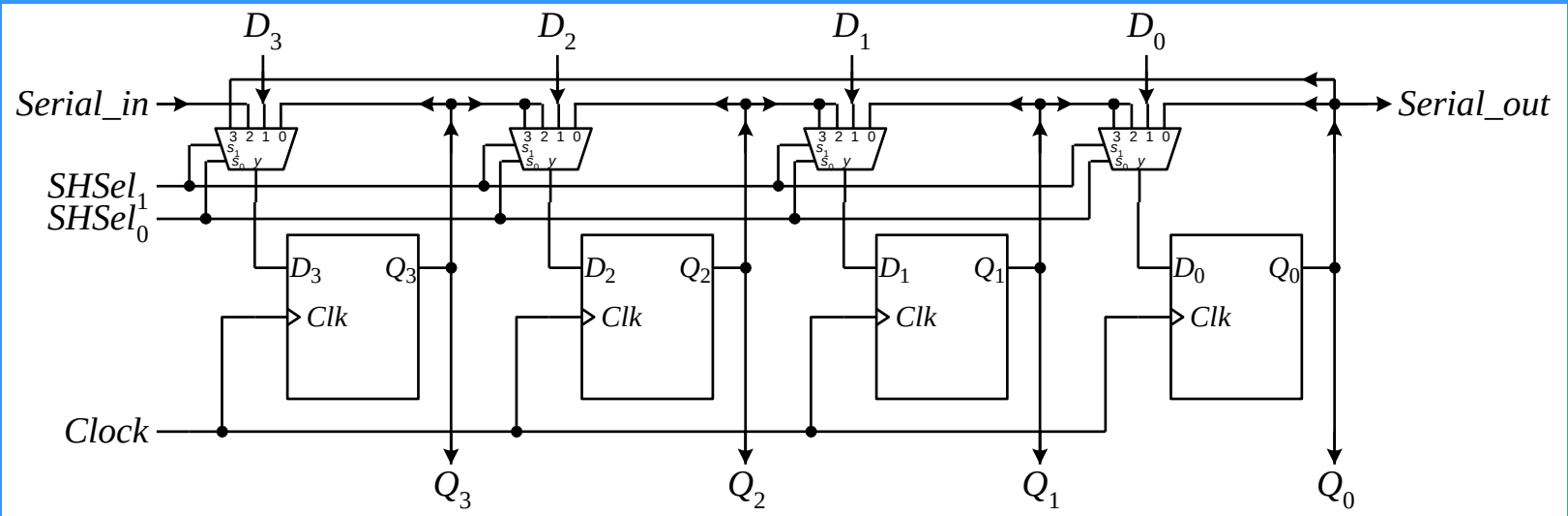


Left-shift register also easy to design

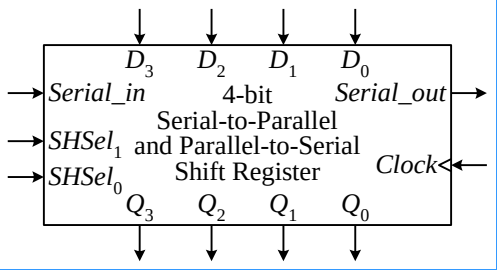


Shift Register

- For serial-to-parallel and parallel-to-serial data conversion



$SHSel_1$	$SHSel_0$	Operation
0	0	No operation (i.e., retain current value)
0	1	Parallel load in new value
1	0	Shift right
1	1	Rotate right

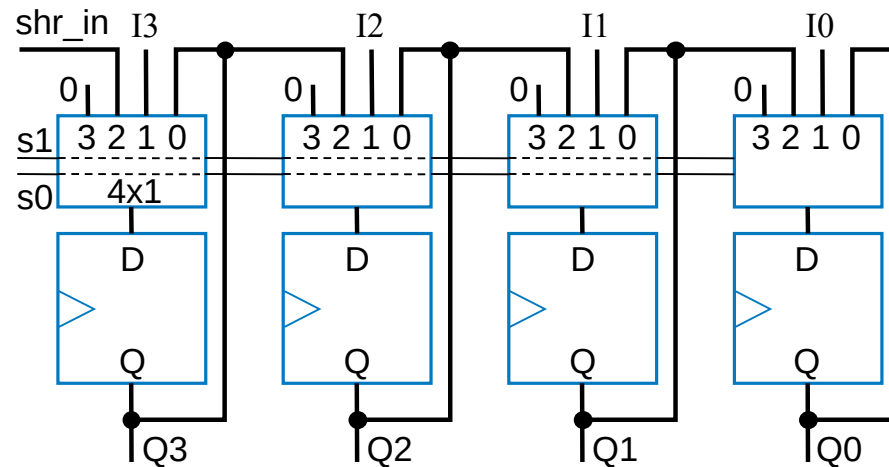


Multifunction Registers

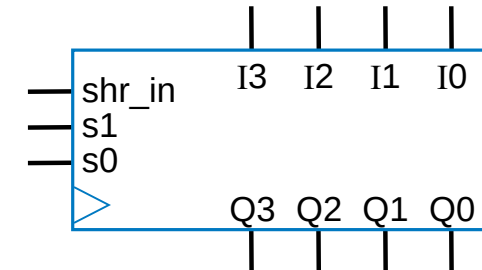
- Many registers have multiple functions
 - Load, shift, clear (load all 0s)
 - And retain present value, of course
- Easily designed using muxes
 - Just connect each mux input to achieve desired function

Functions:

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	(unused - let's load 0s)



(a)

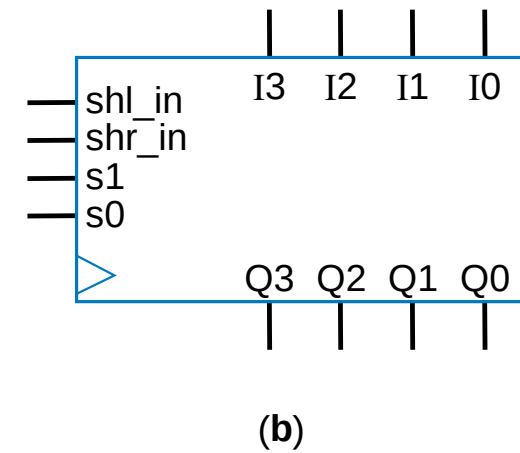
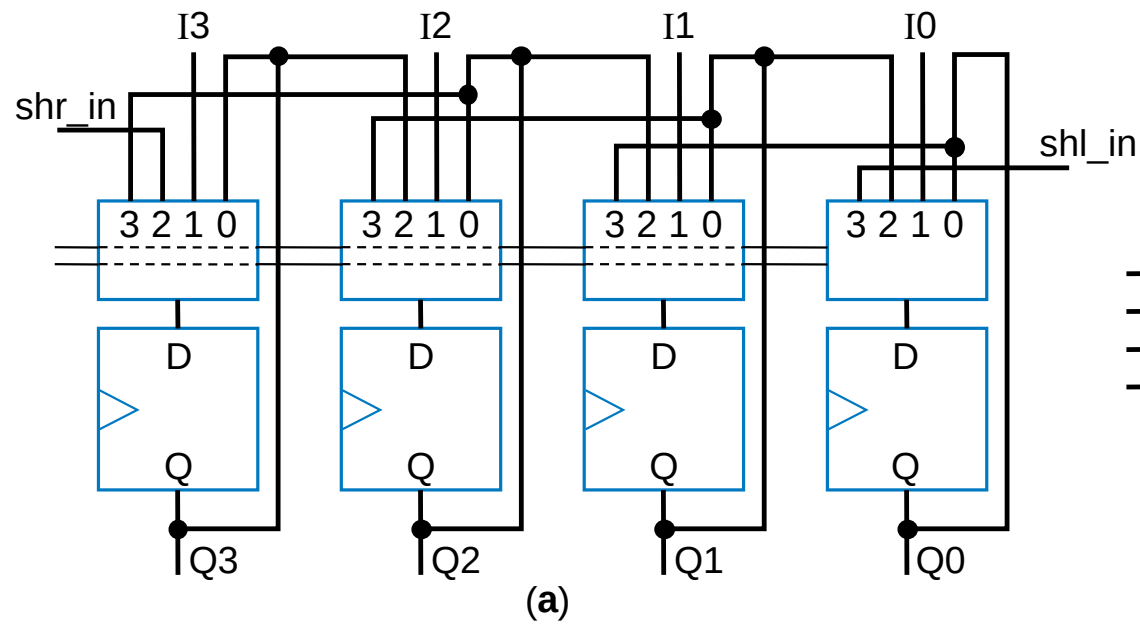


(b)



Multifunction Registers

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	Shift left



Quiz 5.2

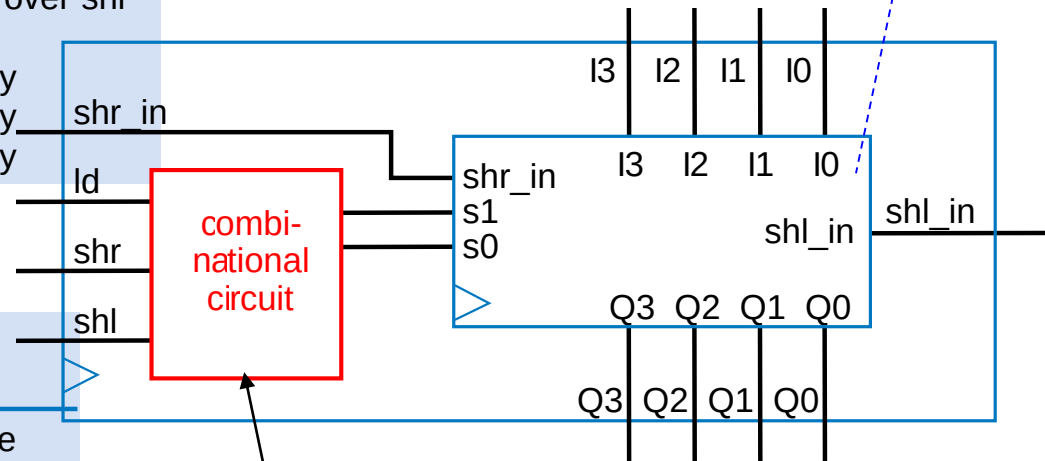
Multifunction Registers with Separate Control Inputs

ld	shr	shl	Operation
0	0	0	Maintain present value
0	0	1	Shift left
0	1	0	Shift right
0	1	1	Shift right – shr has priority over shl
1	0	0	Parallel load
1	0	1	Parallel load – ld has priority
1	1	0	Parallel load – ld has priority
1	1	1	Parallel load – ld has priority

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	Shift left

Truth table for combinational circuit

Inputs			Outputs		Note Operation
ld	shr	shl	s1	s0	
0	0	0	0	0	Maintain value
0	0	1	1	1	Shift left
0	1	0	1	0	Shift right
0	1	1	1	0	Shift right
1	0	0	0	1	Parallel load
1	0	1	0	1	Parallel load
1	1	0	0	1	Parallel load
1	1	1	0	1	Parallel load



$$s1 = ld' * shr' * shl + ld' * shr * shl' + ld' * shr * shl$$

$$s0 = ld' * shr' * shl + ld$$



Register Operation Table

- Register operations typically shown using compact version of table
 - X means same operation whether value is 0 or 1
 - One X expands to two rows
 - Two Xs expand to four rows
 - Put highest priority control input on left to make reduced table simple

Inputs			Outputs		Note Operation
ld	shr	shl	s1	s0	
0	0	0	0	0	Maintain value
0	0	1	1	1	Shift left
0	1	0	1	0	Shift right
0	1	1	1	0	Shift right
1	0	0	0	1	Parallel load
1	0	1	0	1	Parallel load
1	1	0	0	1	Parallel load
1	1	1	0	1	Parallel load

ld	shr	shl	Operation
ld	shr	shl	
0	0	0	Maintain value
0	0	1	Shift left
0	1	X	Shift right
1	X	X	Parallel load



Register Design Process

- Can design register with desired operations using simple four-step process

TABLE 4.1 Four-step process for designing a multifunction register.

	Step	Description
1.	<i>Determine mux size</i>	Count the number of operations (don't forget the maintain present value operation!) and add in front of each flip-flop a mux with at least that number of inputs.
2.	<i>Create mux operation table</i>	Create an operation table defining the desired operation for each possible value of the mux select lines.
3.	<i>Connect mux inputs</i>	For each operation, connect the corresponding mux data input to the appropriate external input or flip-flop output (possibly passing through some logic) to achieve the desired operation.
4.	<i>Map control lines</i>	Create a truth table that maps external control lines to the internal mux select lines, with appropriate priorities, and then design the logic to achieve that mapping



Register Design Example

- Desired register operations
 - Load, shift left, synchronous clear, synchronous set
 - Want unique control input for each operation

Step 1: Determine mux size

5 operations: above, plus maintain present value (don't forget this one!)
 → Use **8x1 mux**

Step 2: Create mux operation table

Step 3: Connect mux inputs

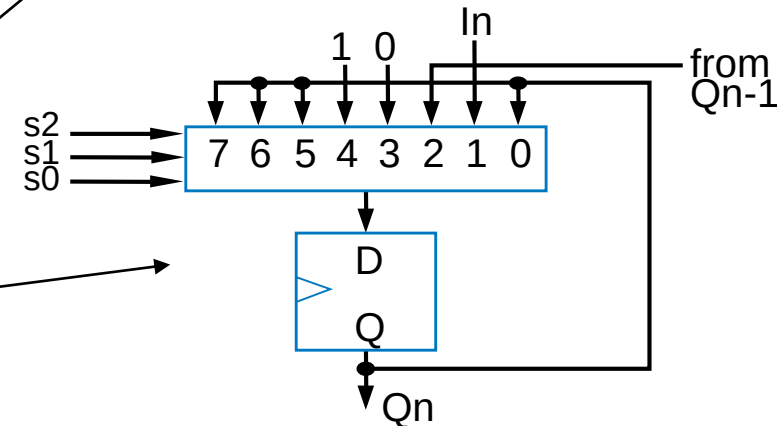
Step 4: Map control lines

$$s2 = \text{clr}' * \text{set}$$

$$s1 = \text{clr}' * \text{set}' * \text{ld}' * \text{shl} + \text{clr}$$

$$s0 = \text{clr}' * \text{set}' * \text{ld} + \text{clr}$$

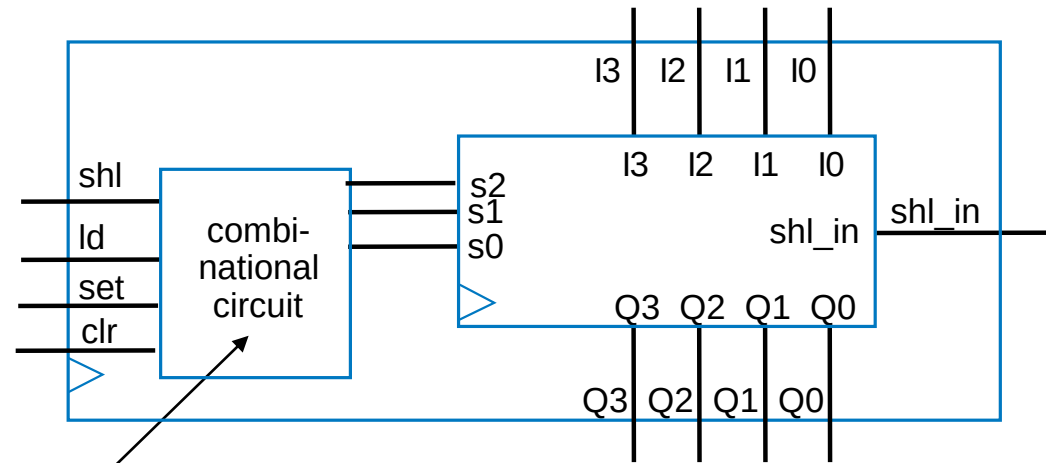
s2	s1	s0	Operation
0	0	0	Maintain present value
0	0	1	Parallel load
0	1	0	Shift left
0	1	1	Synchronous clear
1	0	0	Synchronous set
1	0	1	Maintain present value
1	1	0	Maintain present value
1	1	1	Maintain present value



Inputs				Outputs			Operation
clr	set	ld	shl	s2	s1	s0	
0	0	0	0	0	0	0	Maintain present value
0	0	0	1	0	1	0	Shift left
0	0	1	X	0	0	1	Parallel load
0	1	X	X	1	0	0	Set to all 1s
1	X	X	X	0	1	1	Clear to all 0s



Register Design Example



Step 4: Map control lines

$$s2 = \text{clr}' * \text{set}$$

$$s1 = \text{clr}' * \text{set}' * \text{ld}' * \text{shl} + \text{clr}$$

$$s0 = \text{clr}' * \text{set}' * \text{ld} + \text{clr}$$

Inputs				Outputs			Operation
clr	set	ld	shl	s2	s1	s0	
0	0	0	0	0	0	0	Maintain present value
0	0	0	1	0	1	0	Shift left
0	0	1	X	0	0	1	Parallel load
0	1	X	X	1	0	0	Set to all 1s
1	X	X	X	0	1	1	Clear to all 0s



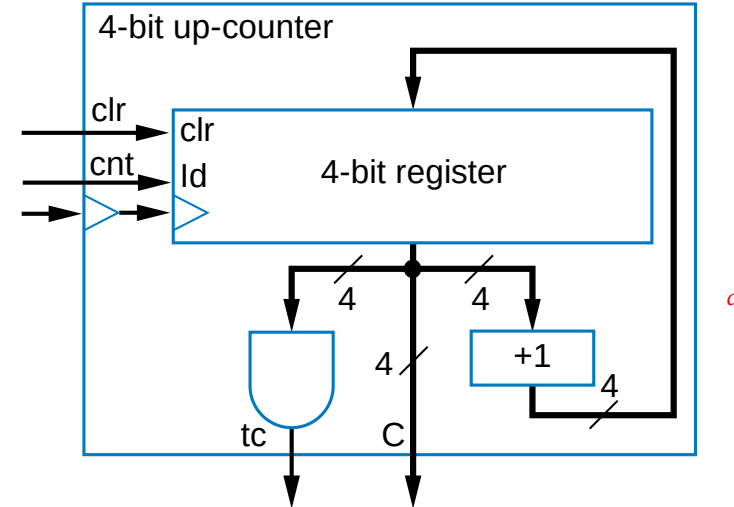
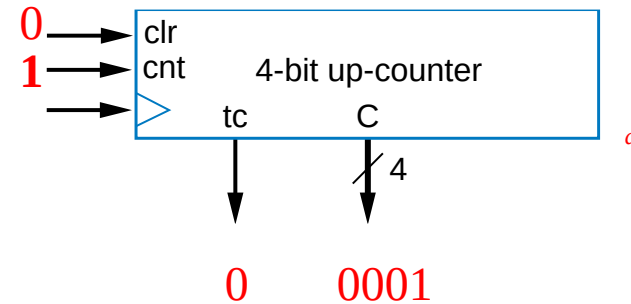
Quiz 5.3

Counters

- Counters: counting a sequence of values
- Different Type
 - The total number of count value
 - The sequence of values that it outputs
 - Whether it counts up or down
 - etc.
- Typical: Binary up counter

Counters and Timers

- ***N-bit up-counter***: N-bit register that can increment (add 1) to its own value on each clock cycle
 - 0000, 0001, 0010, 0011,, 1110, 1111, 0000
 - Count “rolls over” from 1111 to 0000
 - Terminal (last) count, tc, equals 1 during value just before rollover
- Internal design
 - Register, incrementer, and N-input AND gate to detect terminal count

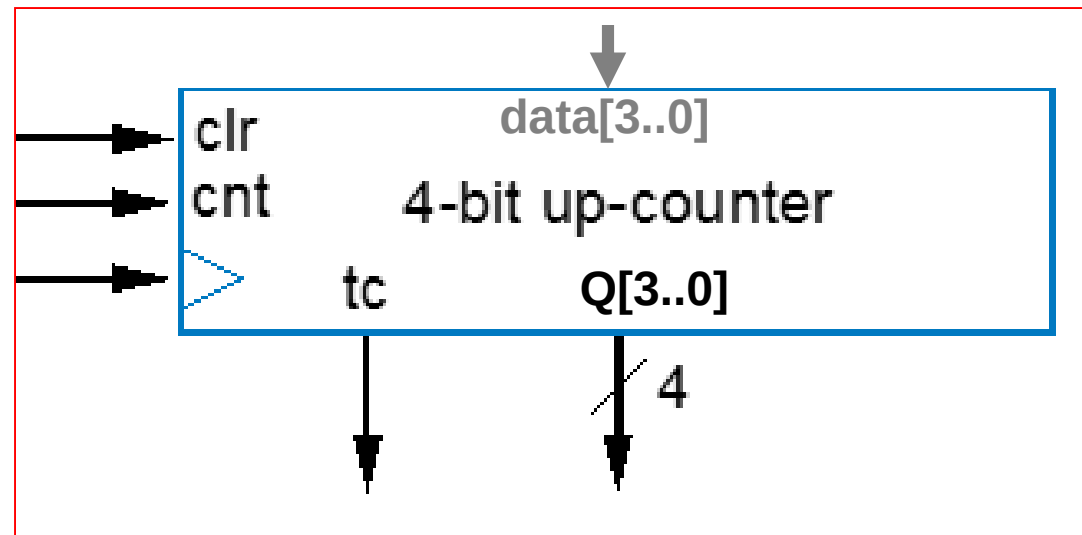


四位元上數計數器

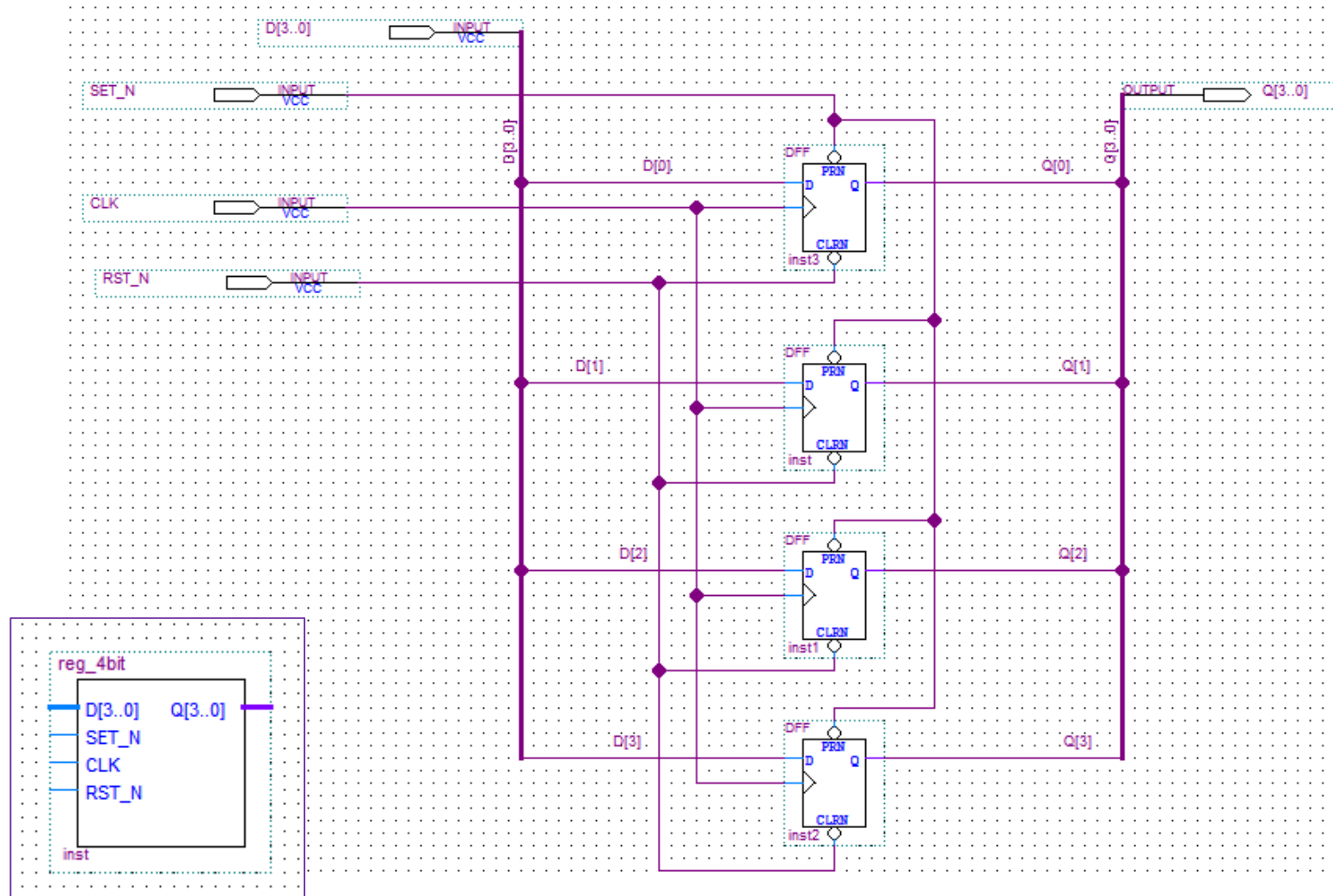


4-bit up counter

- 設計一四位元可清除也可暫停的上數計數器，data為計數上限值，當 $Q = \text{data}$ 時， $tc = 1$ 。

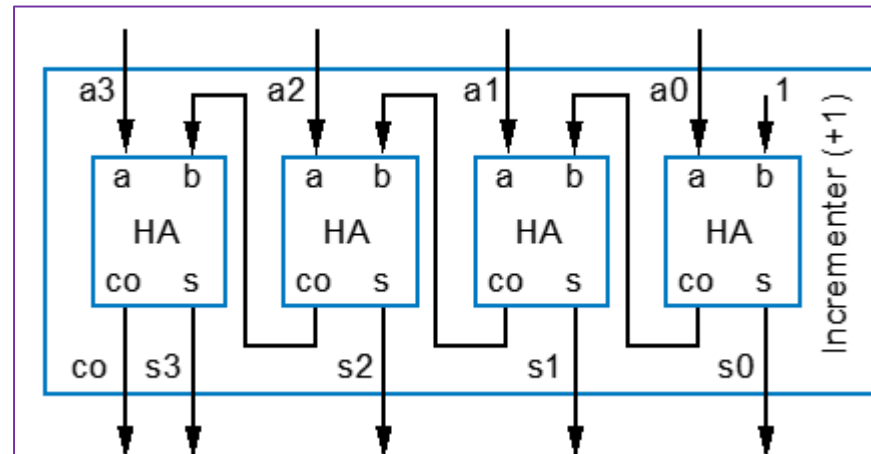
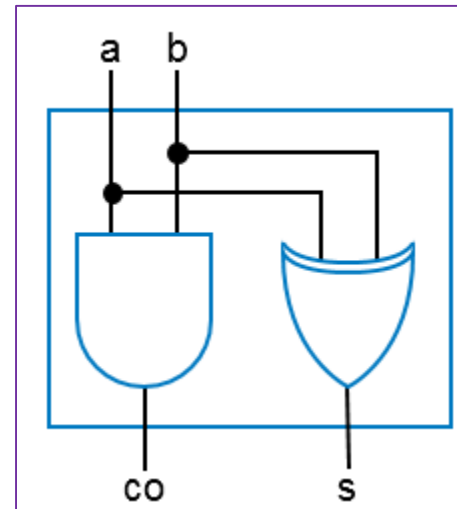


4位元暫存器

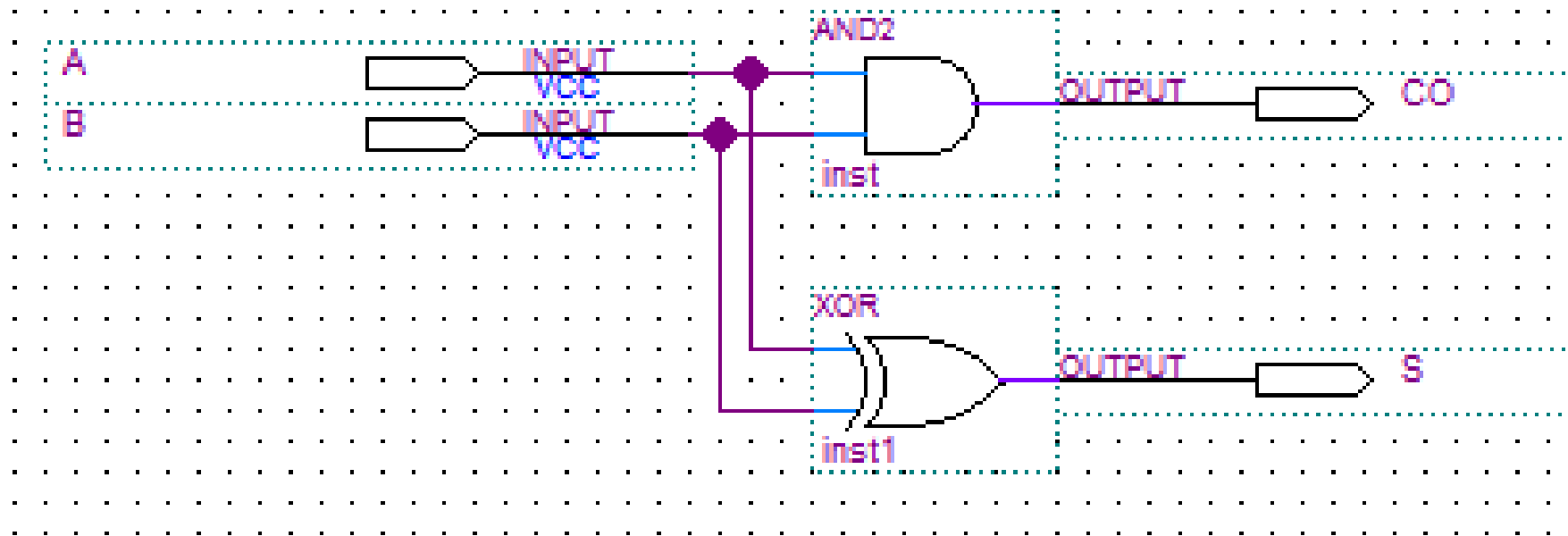


半加器(HA)

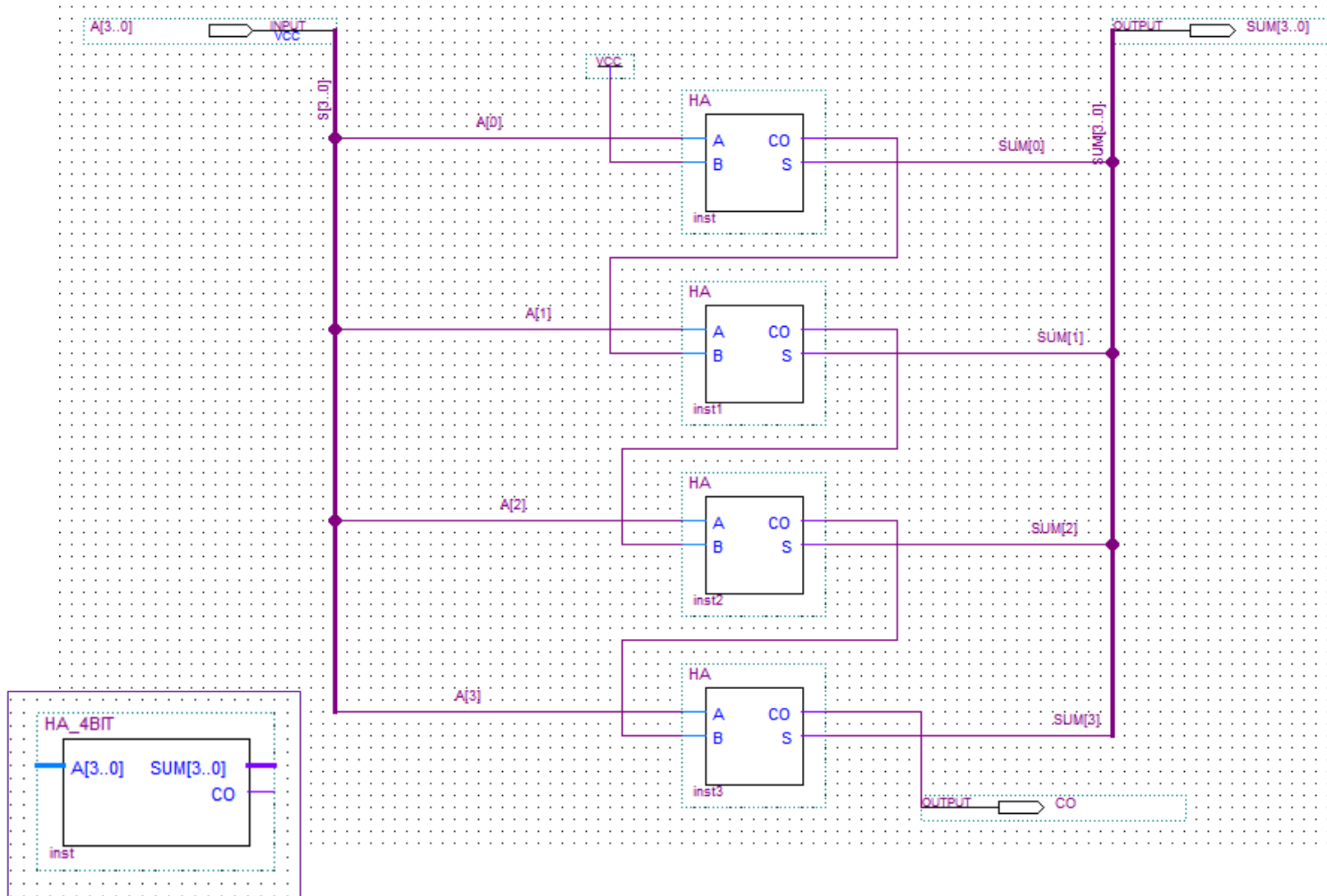
Inputs		Outputs	
a	b	co	s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



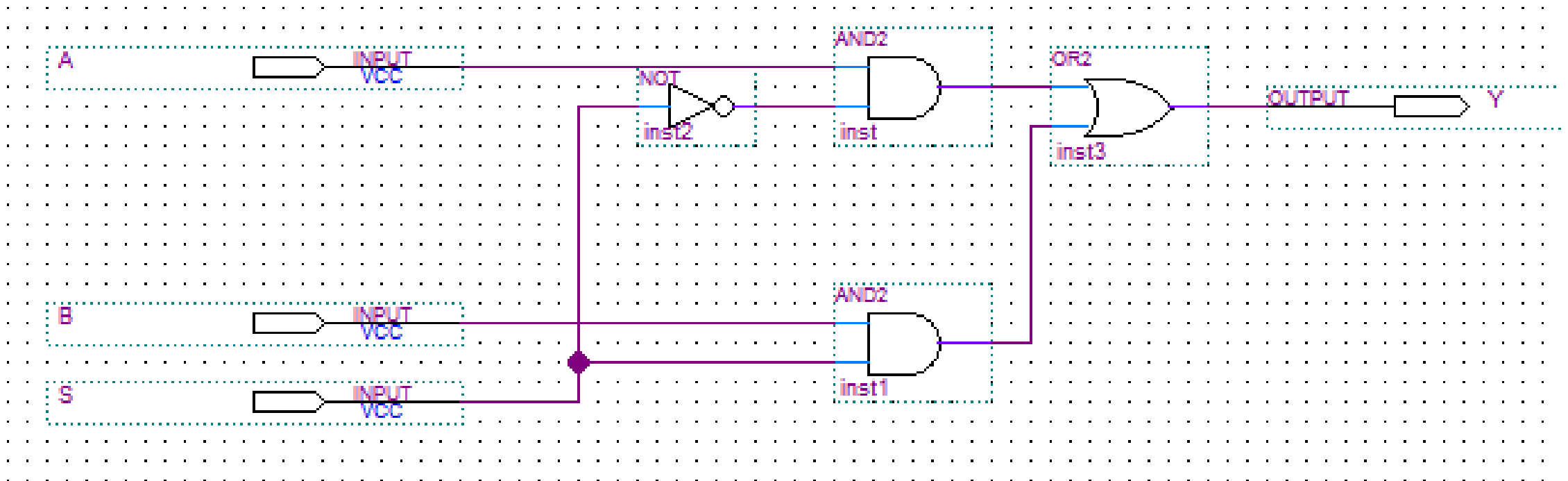
半加器



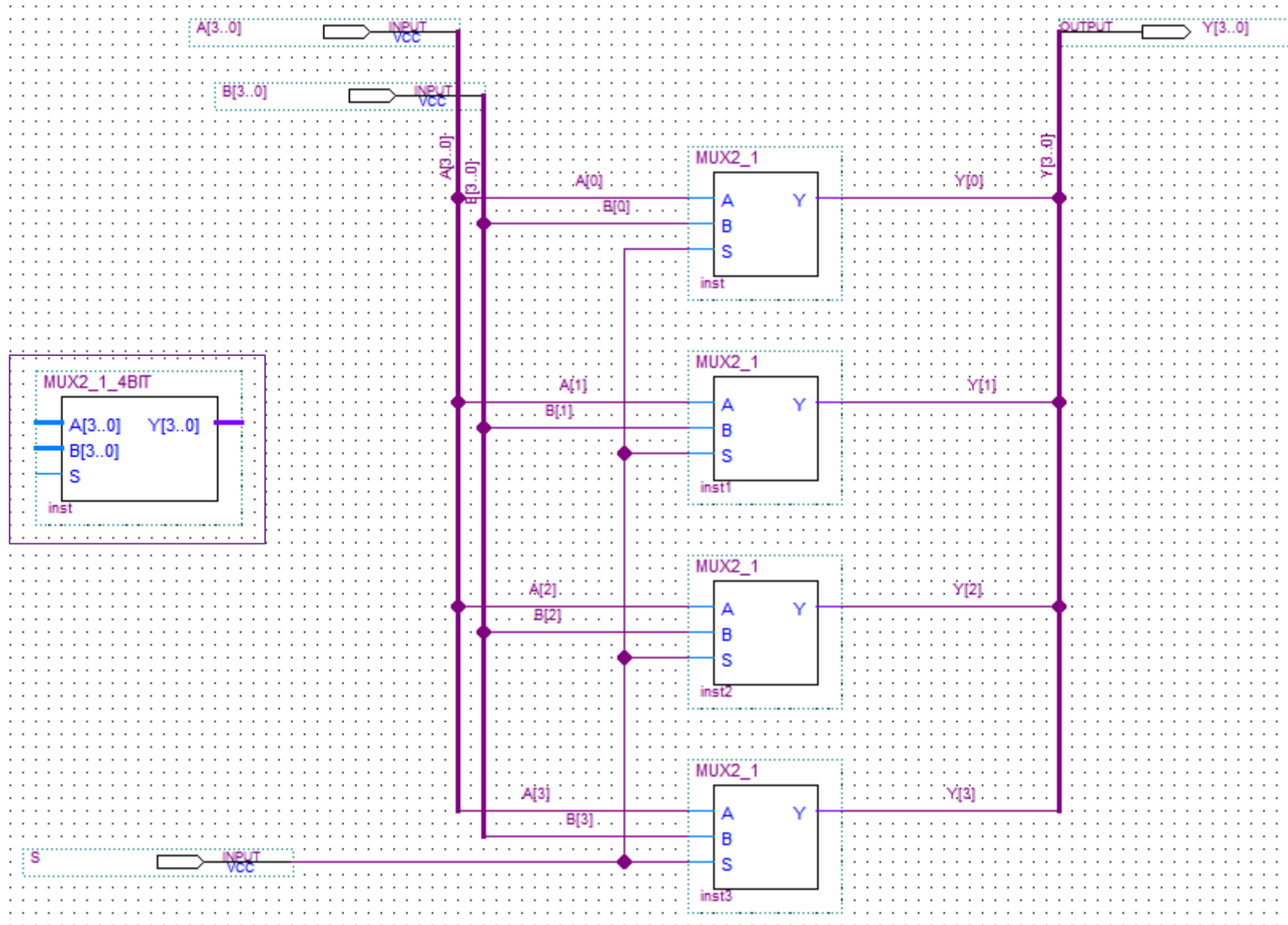
四位元遞增器



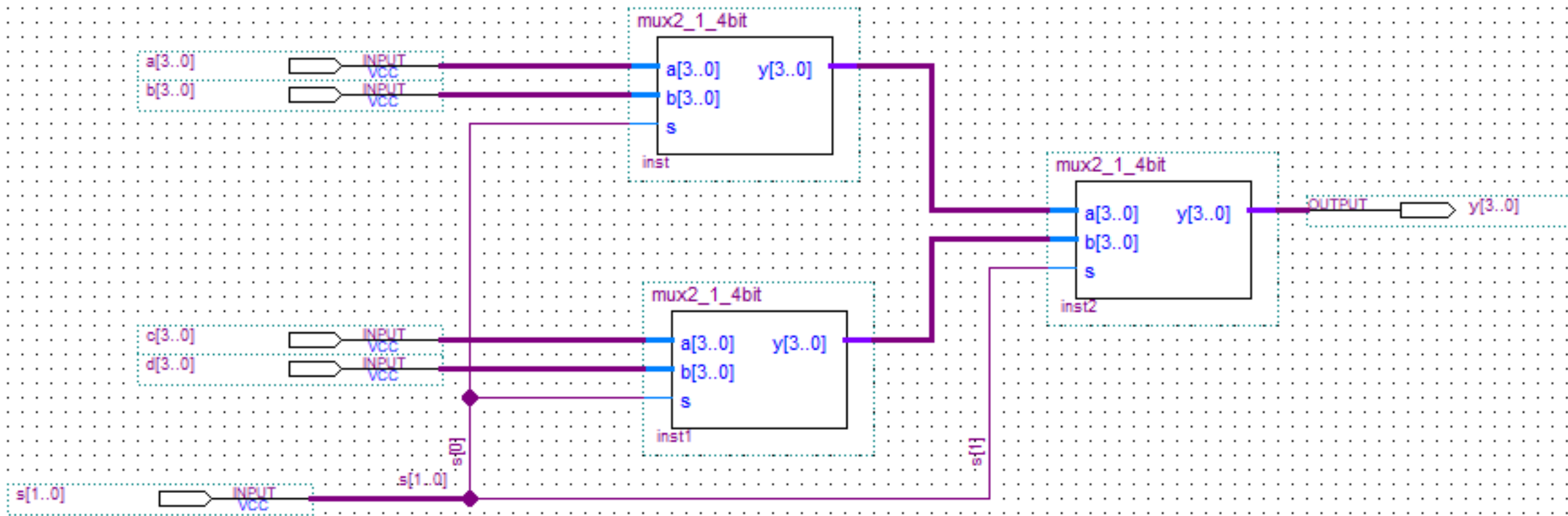
多工器2_1



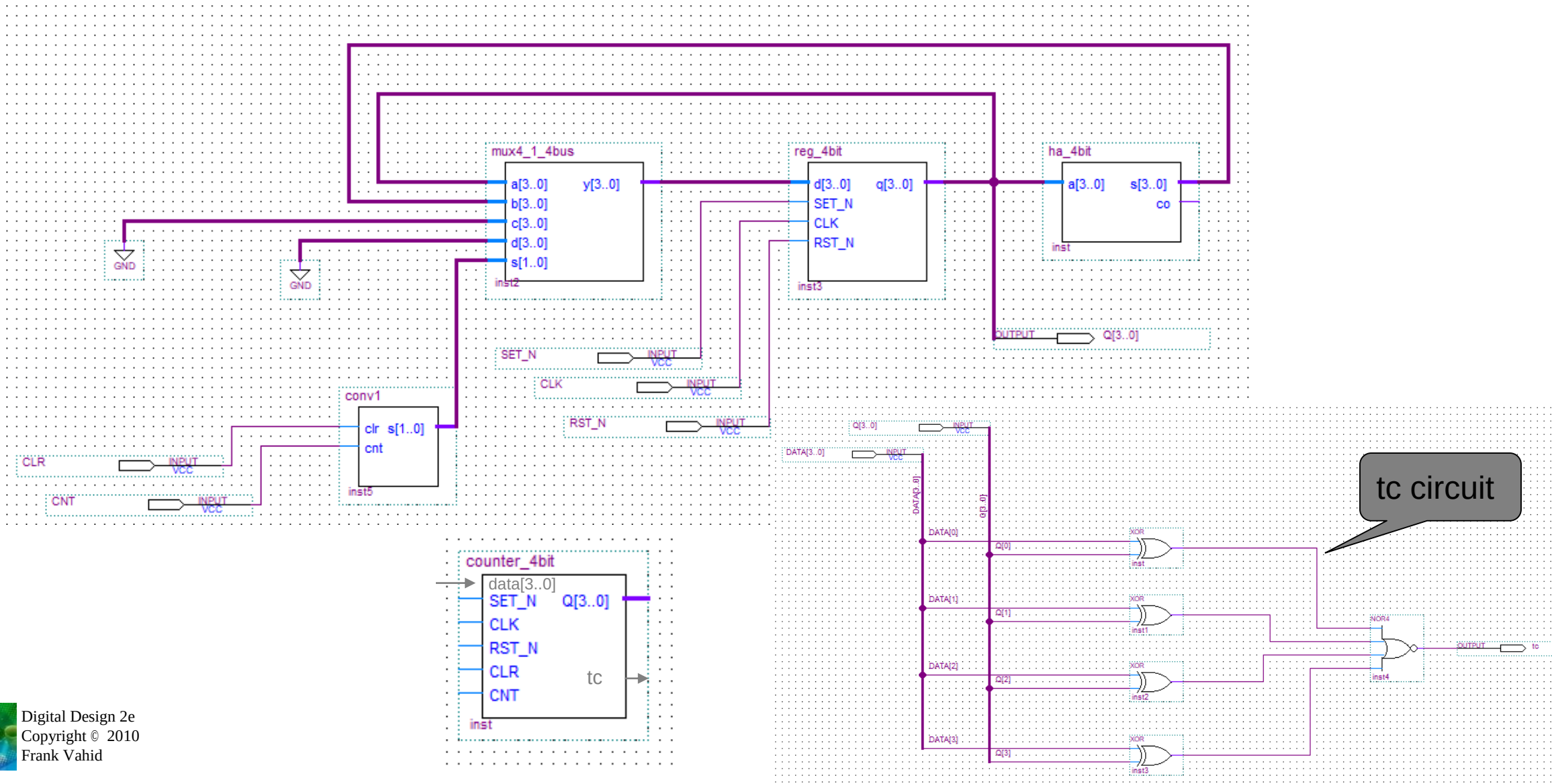
多工器2_1_4BIT



多工器4_1_4BIT

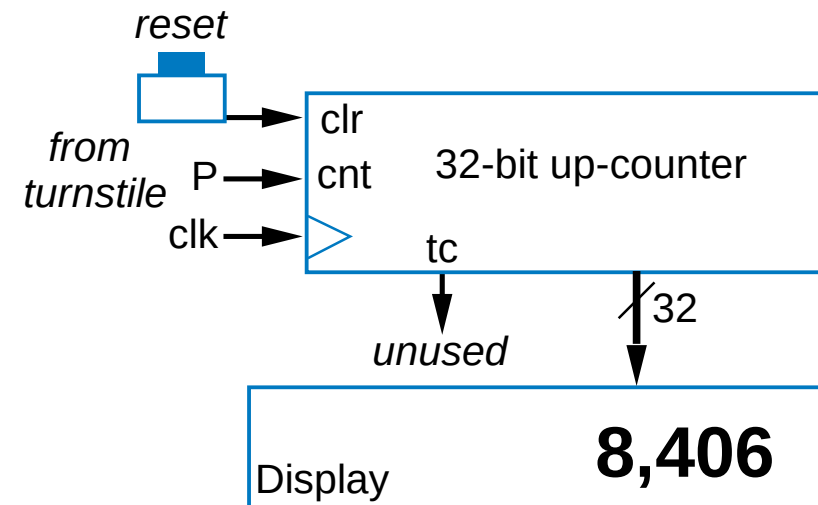


四位元上數計數器(不變/清除/計數)



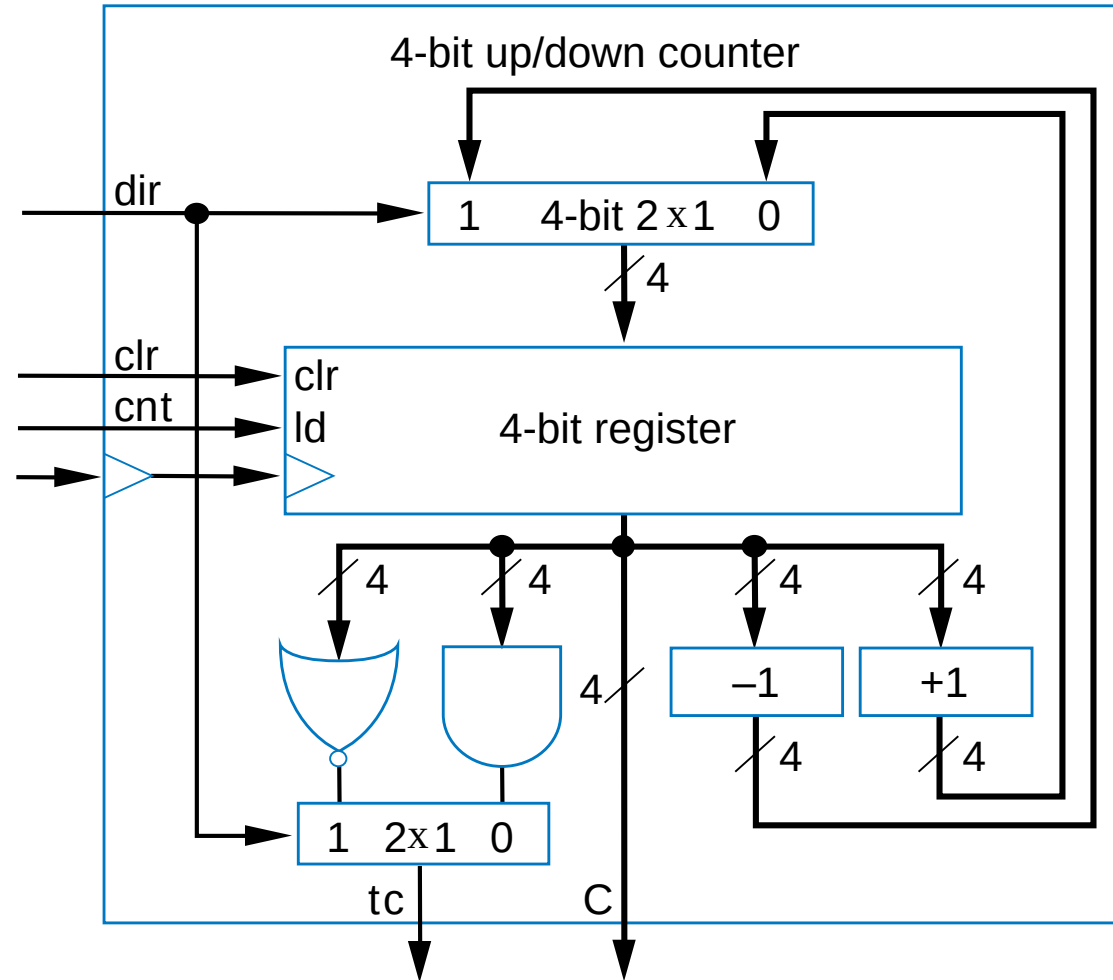
Counter Example: Turnstile with Display

- Count people passing through a turnstile
 - Assume turnstile generates 1-clock pulse on P
 - Connect count to display
 - Button can reset count
- Simple solution using up-counter



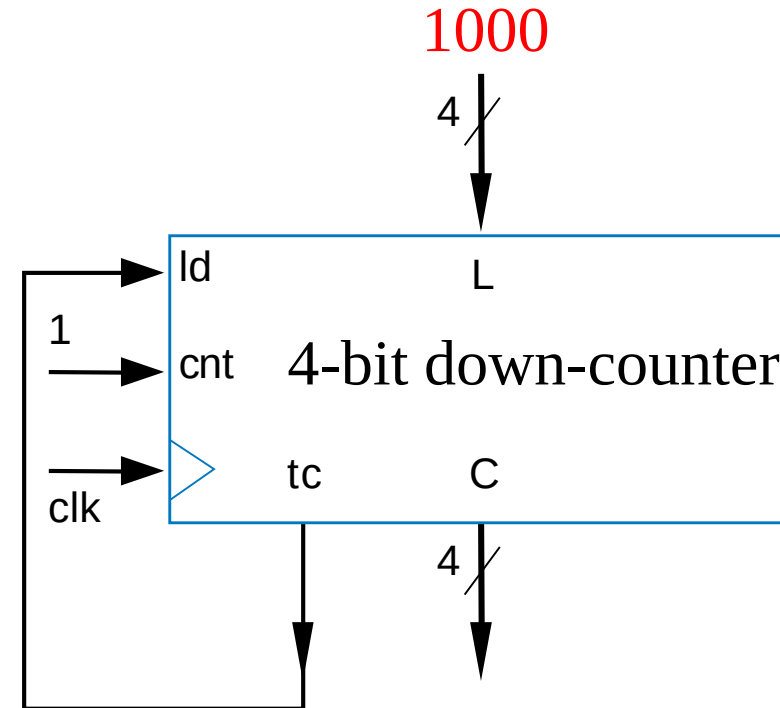
Up/Down-Counter

- Can count either up or down
 - Includes both incrementer and decrementer
 - Use dir input to select, via 2x1 mux: dir=0 means up
 - Likewise, dir selects appropriate terminal count value (all 1s or all 0s)



Counter with Parallel Load

- Useful to create pulses at specific multiples of clock
 - Not just at N-bit counter's natural wrap-around of 2^N
- Example: Pulse every 9 clock cycles
 - Use 4-bit down-counter with parallel load
 - Set parallel load input to 8 (1000)
 - Use terminal count to reload
 - When count reaches 0, next cycle loads 8.
 - Why load 8 and not 9? Because 0 is included in count sequence:
 - 8, 7, 6, 5, 4, 3, 2, 1, 0 → 9 counts

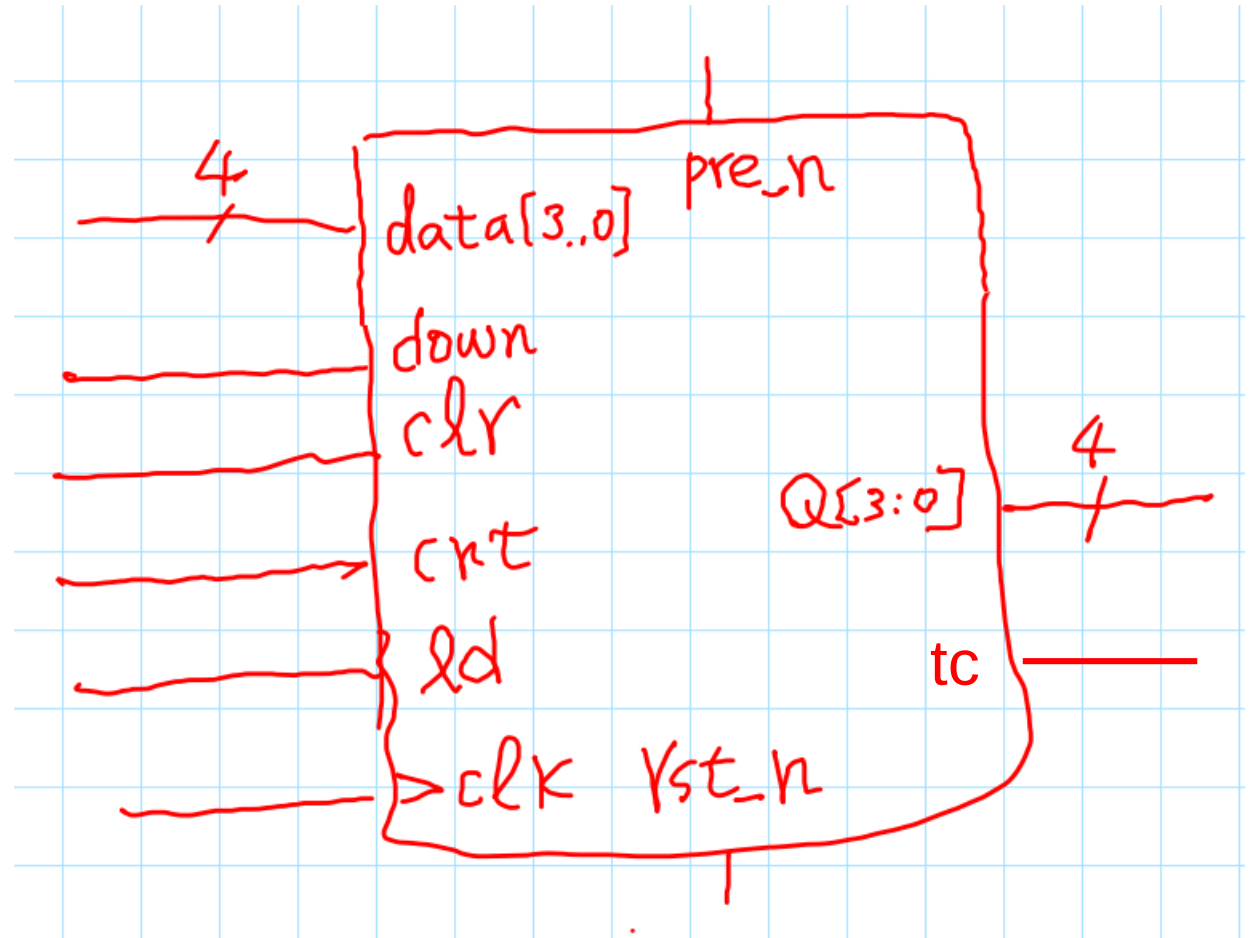


四位元上/下數計數器



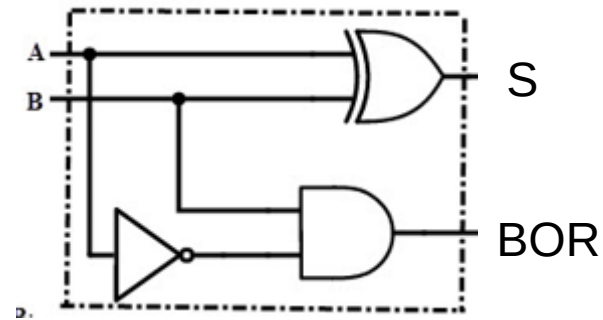
Lab5-2

- 設計一四位元上/下數計數器
- 功能：
 - 上/下數
 - 清除
 - 計數
 - 載入

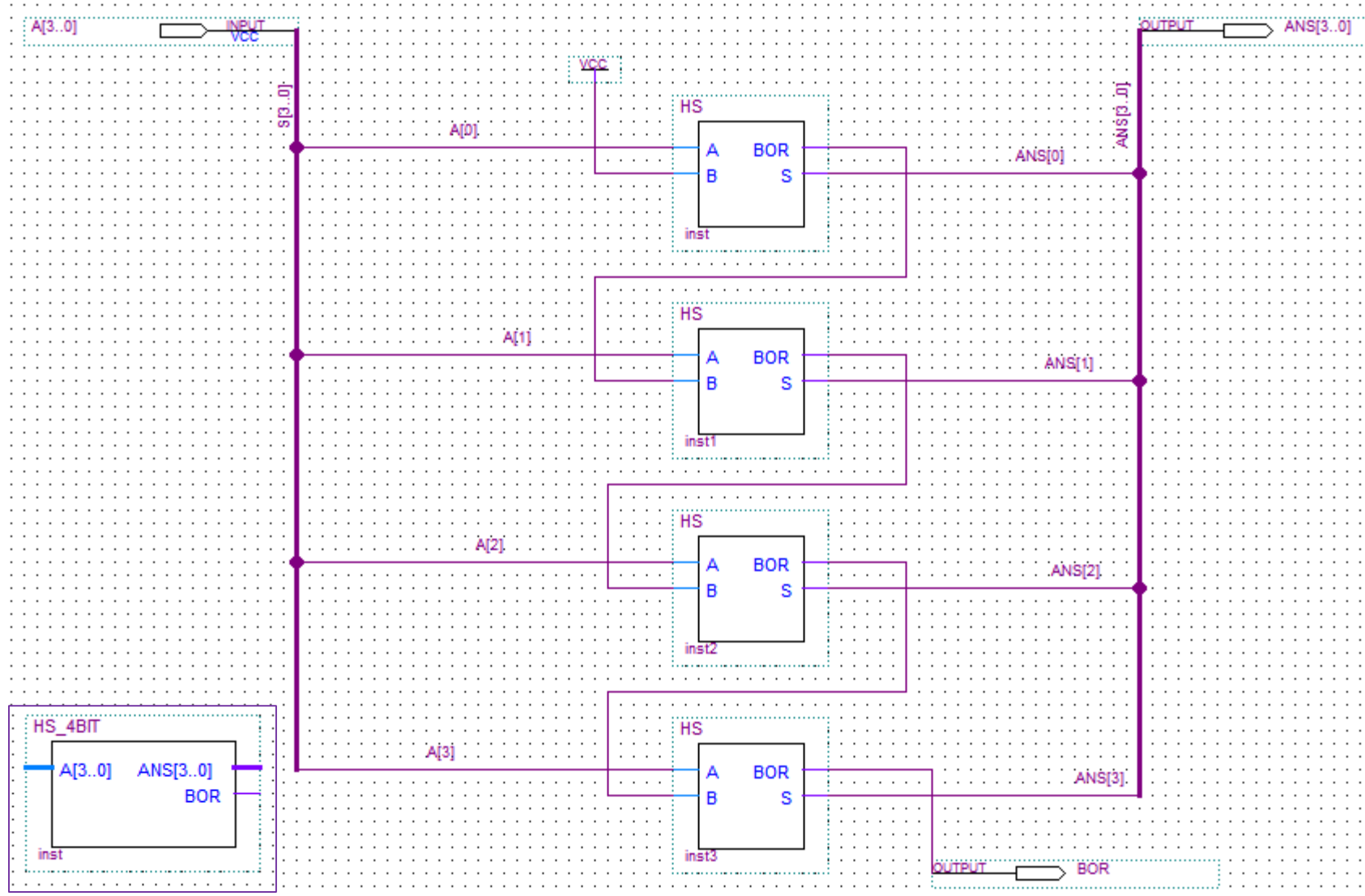


半減器(HS)

A	B	BOR	S
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0



四位元遞減器



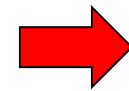
提示

clr	ld	cnt	down	S[2]	S[1]	S[0]
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	1
0	0	1	1	0	1	0
0	1	X	X	1	0	0
1	X	X	X	0	1	1

S[2]	S[1]	S[0]	Q _{n+1}
0	0	0	Q _n
0	0	1	Q _n +1
0	1	0	Q _n -1
0	1	1	0
1	0	0	data
1	0	1	Q _n
1	1	0	Q _n
1	1	1	Q _n



clr	ld	cnt	down	S[2]	S[1]	S[0]
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	1
0	0	1	1	0	1	0
0	1	X	X	1	0	0
1	X	X	X	0	1	1



$$S[2] = \overline{\text{clr}} \cdot \text{ld}$$

$$S[1] = \overline{\text{ld}} \cdot \text{cnt} \cdot \text{down} + \text{clr}$$

$$S[0] = \overline{\text{ld}} \cdot \overline{\text{cnt}} \cdot \overline{\text{down}} + \text{clr}$$

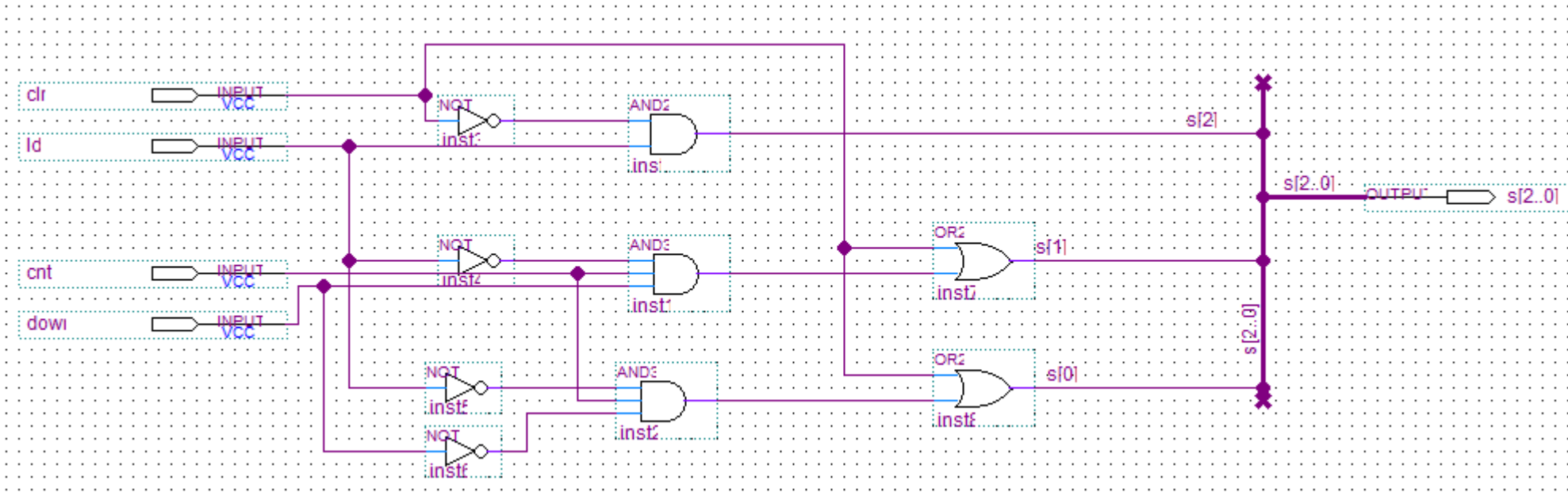


提示

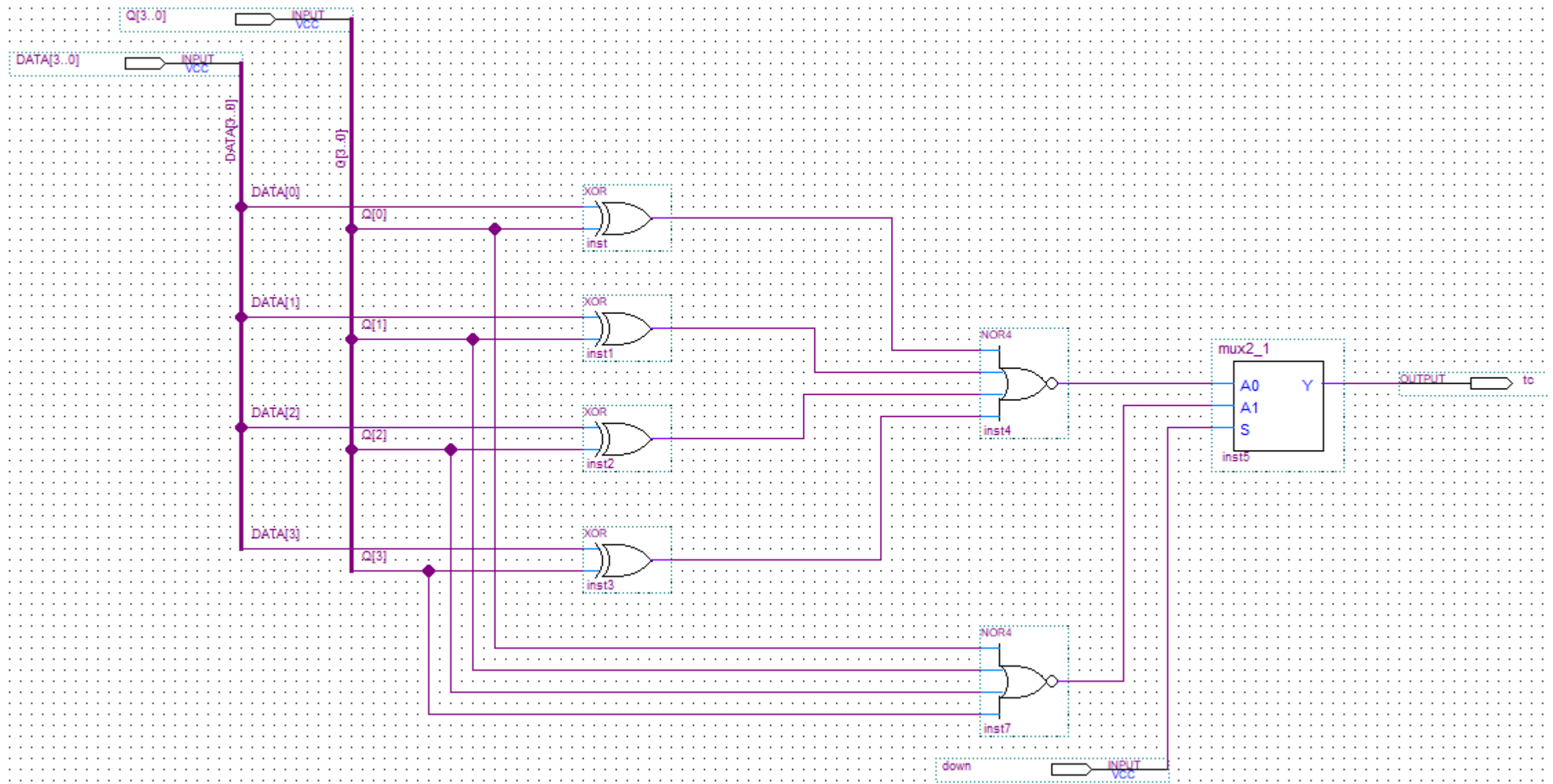
$$s[2] = \overline{clr} \cdot ld$$

$$s[1] = \overline{ld} \cdot cnt \cdot down + clr$$

$$s[0] = \overline{ld} \cdot cnt \cdot \overline{down} + clr$$

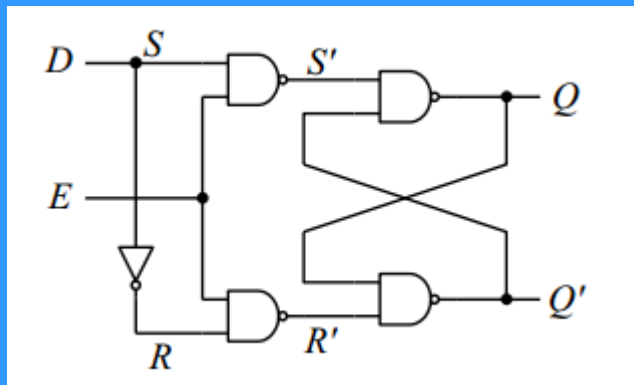


tc circuit

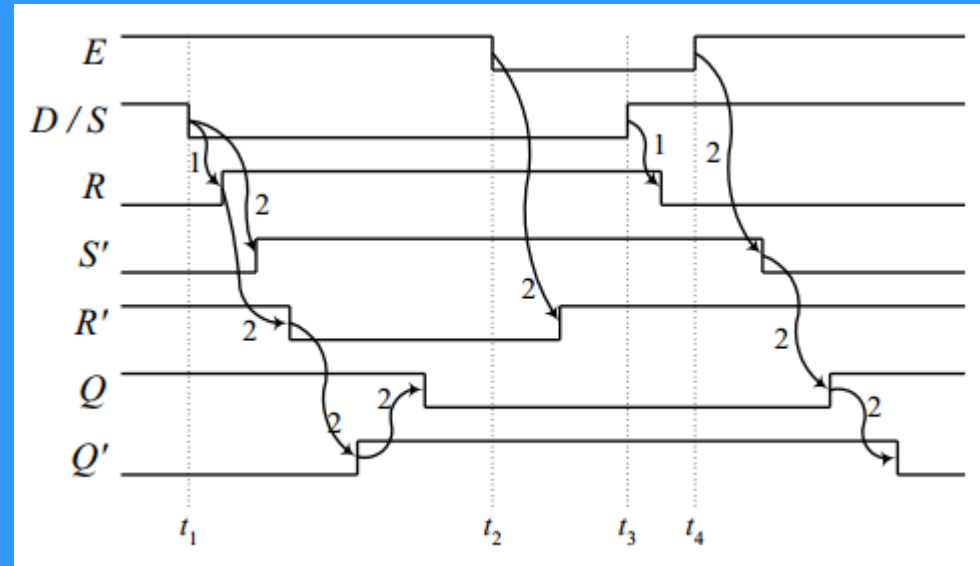


*Timing Issues

- D Latch with enable
 - (a) circuit
 - (b) timing diagram with delays

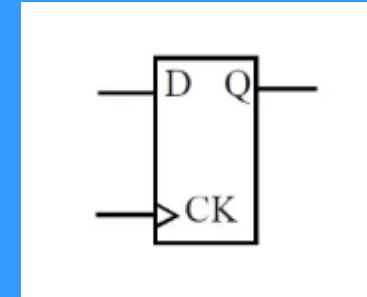


(a)



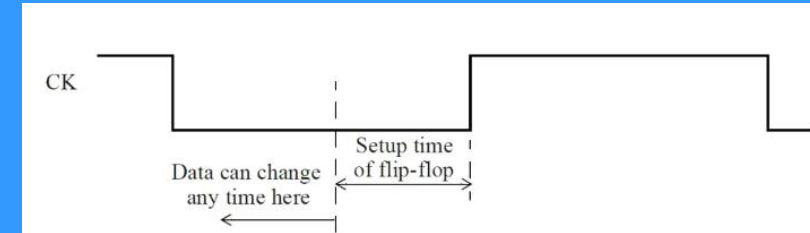
(b)

*Timing Issues



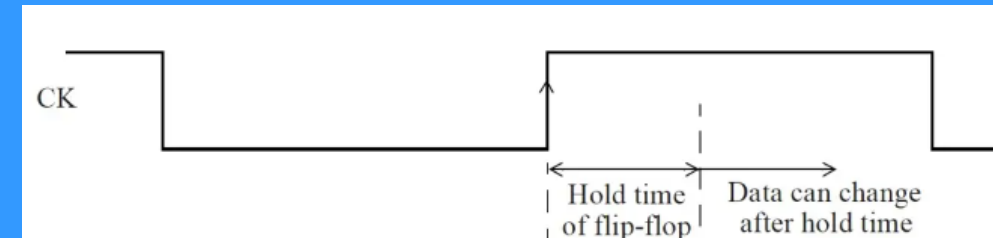
■ Setup time

- 是指在時鐘有效緣（下圖為上升緣）之前，數據輸入端信號必須保持穩定的最短時間



■ Hold time

- 指在時鐘有效緣（下圖為上升緣）之後，數據輸入端信號必須保持穩定的最短時間。



Conclusions

- Latches and Flip-Flops
- Description of a Flip-Flop
 - Characteristic Table
 - Characteristic Equation
 - State Diagram
- Register and Register Files
- Memories
- Shift Registers
- Counters